

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_cb1d52fa-fed\agent-ab78c4c9de0d12f42.jsonl`  
- SHA-256: `745ac5f53229fe48506eb846057ba6c95b86ef2bf5e22561d850c79481958ceb`  
- Source modified: `2026-06-14T08:27:36+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_cb1d52fa-fed`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-14T08:24:06.515Z)

THE PIVOT under study = "PERSONALIZATION-FIRST": instead of one universal model for all users, ship a BATTERIES-INCLUDED baseline that CONTINUALLY SELF-OPTIMIZES to a SINGLE user's own footage. As the user's corpus grows (10-15 videos), the on-device model + PER-USER CUE SELECTION (driven by the EXISTING LOVO / A-B experiment harness run on the user's OWN videos, with the user's CORRECTION LOOP as the per-user label source) becomes heavily tuned for THAT user ? deep analysis of THEIR rallies + shot types (serves, smashes, drives, drop shots). Per-user decisions like "the motion/person cue gives no F1 win for THIS user -> default it OFF for them." The model is optimized for the user's videos, NOT others (wins may hold anecdotally on other footage but weaker). Periodic "BATTERIES UPGRADES" improve the shipped baseline so it needs merely <5 videos to reach superior quality. Two user archetypes: (a) an economically-generous hobbyist who records sessions some days; (b) a professional with 100+ hours of their own video. CONTRAST = "generalization-first" (build one model that works on everyone's videos). The owner believes the harness + eval + A/B were DESIGNED to enable this roadmap.

DIMENSION: TECHNICAL FEASIBILITY of per-user continual personalization. Read the repo's owned-model plan (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) + the rally-detection design (docs/HOW_RALLY_DETECTION_WORKS.md, docs/MULTI_SIGNAL_FUSION_PLAN.md). Analyze: the COLD-START (a brand-new user with <5 videos relies on the baseline); what actually PERSONALIZES cheaply on-device ? the lightweight Gen-0/Gen-1 learned head + per-user cue/threshold selection on the frozen WASB shuttle substrate, vs the heavyweight Qwen3-VL (on-device fine-tuning of a VLM is likely infeasible on a hobbyist's machine); per-user LOVO/A-B on TINY per-user corpora (the small-N statistics problem, same as the n=6 issue but per user); catastrophic forgetting across "batteries upgrades". Web-ground on-device/continual/online personalization + small-data fine-tuning. Return pros/cons + what's feasible-now vs needs-research.

2. assistant (2026-06-14T08:24:13.267Z)

I'll analyze the technical feasibility of per-user continual personalization. Let me start by reading the key planning documents and understanding the existing harness/eval/A-B infrastructure.

3. user (2026-06-14T08:24:15.383Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-14T08:24:15.391Z)

```
1      # How rally detection works (the 2-layer mental model)  
2  
3      A short conceptual map of how we go from raw video to rally `[start, end]` segments ?  
4      and, crucially, **which layer our annotation can improve.**  
5  
6      ```  
7      video ??? frames ??? [DETECTION] ??? shuttle (x,y,visible) per frame
```

```

8          ?          ?
9          (WASB CNN,      per-frame trajectory
10         frozen weights)  ?
11                           ?
12         velocity signal  (how fast the shuttle moves)
13                           ?
14         [SEGMENTATION]  ??? the 3 knobs we tune
15                           ?
16                           ?
17         rally windows  [start, end] ??? IoU vs golden
labels
18     ...
19
20     ## Two layers ? keep them separate in your head
21
22     **1. DETECTION ? "where is the shuttle, this frame?"**
23     WASB is a CNN that, for each frame, outputs a heatmap ? the shuttle's `(x, y)` + a
24     visibility/confidence. Its weights were learned (by gradient descent, on shuttle-
coordinate
25     labels) and we use them **frozen**. We chose a *shuttle* detector on purpose: "shuttle
in
26     flight" is the signal for "a rally is happening," and it's **camera-agnostic** ? unlike
27     player-motion, which fooled the old heuristic on broadcast cameras that pan with the
players.
28     (Your videos are static tripod, so player-motion would also work ? but shuttle-motion is
the
29     robust general bet.)
30
31     **2. SEGMENTATION ? "when is a rally?"**
32     From the per-frame `(x, y)` we compute the shuttle's **velocity** (displacement between
frames).
33     Then three knobs turn that into rally intervals:
34     - **`velocity_thresh`** ? a frame is "active" only if the shuttle is moving faster than
this
35     (in flight, not lying on the floor / being picked up).
36     - **`merge_gap`** ? active frames within this many seconds get joined into one rally
(bridges
37     brief gaps: a shuttle momentarily occluded, or the beat between two hits).
38     - **`min_window_duration`** ? throw away windows shorter than this (blips / false
flickers).
39
40     Output: a list of `[start, end]` windows = the rallies.
41
42     ## Where the annotation fits (the key point)
43     Our Roora/golden labels are **rally `[start, end]` times** ? they say *when* rallies
are, not
44     *where* the shuttle is*. So they can't teach WASB's detector (that needs `(x,y)` labels).
They
45     **can** grade and tune the **segmentation layer**: we score predicted windows against
the golden
46     rallies with **temporal IoU** (how much they overlap), and pick the 3 knobs that
maximise it ?
47     **calibrate on some rallies, measure on held-out ones** so the number is honest.
48
49     ## Where sparks could land (open questions ? not commitments)
50     Things worth a shower-thought; any of these could beat 3 hand-tuned thresholds:
51     - **Richer features than raw velocity:** acceleration, trajectory *continuity* (a real
rally is a
52     smooth arc that survives occlusion), shuttle *height/arc*, or **direction reversals**
(each
53     reversal ? a shot ? ties to `shots_count`!).
54     - **A tiny learned segmenter** instead of 3 thresholds: feed the trajectory features to
a small
55     model trained on the golden rally labels (still the segmentation layer, just smarter).
56     - **Hysteresis** (two thresholds: enter-rally high, stay-in-rally low) ? robust to mid-

```

rally dips.

```

57     - **Rhythm/periodicity:** rallies have a hit cadence; dead time doesn't.
58     - **Fusion:** combine shuttle-motion (WASB) with player-motion for static cams. ?
designed in
59     [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md) (one rally-fusion layer;
the person cue is
60     ADR-007's parked #2 and `rally_gate.py`'s named-but-unbuilt "Gate B").
61     - **Per-video calibration as a *feature*:** "watch a few labeled rallies ? auto-tune to
this
62     court/lighting ? segment the rest sharply" ? exactly your earlier idea, at the right
layer.
63
64     The whole point of per-video calibration: a handful of *your* labeled rallies nudges
these knobs
65     to *your* court/camera, and we report the IoU lift on the rallies it never saw.
66

```

5. user (2026-06-14T08:24:15.392Z)

```

1     # Owned-model implementation plan (roadmap) ? DRAFT for owner review
2
3     **Status:** PLAN / roadmap. No owned-model *implementation* starts until the owner
greenlights a specific
4     milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a
training source).
5     **Grounded in:** `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO +
learned prototype
6     (`docs/archives/quality-iterations/2026-06-multivideo-lovo/`), and a verified strategy
workflow (2026-06-09).
7
8     ## Design principle (owner directive): *envision now, build later*
9     Every deferred piece (conversational-QA, on-device, multisport, the VLM) must be
**architecturally envisioned
10    now** so we never hit a mammoth refactor. We **design the seams** (data contracts,
export boundaries, the
11    event store) up front and **implement behind them** when each milestone is greenlit.
Punting a *feature* is fine;
12    punting its *seam* is not.
13
14    ## North Star (reframed per PR #61 review)
15    - **Immediate deliverable:** a **training framework + harness** that produces a model
with **very high precision/
16    recall for extracting highlights + rallies from a clip**. *Quality is the first-order
objective.*
17    - **Eventual product:** a **single, sport-agnostic, conversational** video model ?
upload a clip, then ask
18    contextual questions ("longest rally", "make a clip of all service faults", "rallies
over 20 shots").
19    - **On-device is LATER** (a model-compression goal so power users save upload
cost/hassle) ? definitely on the
20    roadmap, but **not** the immediate target. Don't over-index the strategy on it now.
21    - **The moat = the consented dataset + the eval harness, not the weights.**
22
23    ## Hard guardrails (non-negotiable ? "never corrupted")
24    - **Commercial-clean licensing for EVERY dependency ? code, data, AND model weights: MIT
/ Apache-2.0 / BSD only.**
25    (Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-
restrictive: Gemma 3 (viral),
26    **Gemma 3n** (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU
cap), Commons-Clause repos.
27    - **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public
grounding-CoT datasets that
28    were Gemini-generated (e.g. TVG-R1's 56K) ? we must originate our own CoT supervision
(a real, binding cost).
29    - **Exclude minors; per-user training data needs a separate explicit opt-in; split data

```

```

by match.**
30     - ? **Base-model pretraining provenance is unauditale for *every* open VLM (incl.
Qwen3-VL).** Our "no paid-LLM
31     training" rule is enforceable at *our fine-tuning data* layer, not the base's
pretraining ? an **explicit owner
32     risk-acceptance** is required to use any open VLM. (Governed by the base's license,
not our pipeline.)
33
34     ## The model strategy (verified 2026-06-09)
35     **Framework ? base** (the Gemma-4-vs-ms-swift clarification): a *framework* is the forge
(runs SFT/GRPO, emits
36     weights); a *base* is the metal you forge. You pick one framework and feed it any clean
base.
37     - **Framework: ms-swift (Apache-2.0)** ? the one forge that does native **video + audio
+ timestamp-grounding +
38     GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core;
*avoid its AGPL Studio UI*)
39     as the 8 GB-local SFT/compression tool for the deferred on-device goal. ? Point-in-
time snapshot ? re-verify at impl.
40     - **Base (when we reach the VLM): Qwen3-VL (Apache-2.0, uniform across sizes)** ? long-
video context, purpose-built
41     **timestamp emission** (second-level localization), multi-turn video chat. Re-confirm
the exact size's LICENSE
42     pre-release. **Gemma-4 stays AMBER and bench-only** ? its Apache grant is likely but
its Prohibited-Use posture is
43     unconfirmed (counsel needed), *and* its video/timestamp capability is disputed; not
the primary extractor.
44     **InternVL3 (MIT/Apache)** = clean fallback if a Qwen blocker appears. ? base swap is
cheap *only within
45     timestamp-capable Apache bases* (timestamp data-format + grounding-CoT are base-
specific).
46     - **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss
strict temporal boundaries**
47     (best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse
regions, not accurate boundaries").
48     **Boundary accuracy IS the product.** The head (`rally_seq_proto.py`) is already de-
risked on our corpus (held-out
49     AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params
(trivially compressible later),
50     and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So:
**head now ? VLM later**,
51     strictly sequenced. When the VLM comes, prefer **two-stage** (VLM coarse-proposes,
head refines boundaries) over
52     wholesale replacement. *(Open: the VLM-ceiling number is open-domain; our amateur
footage may differ ? re-measure.)*
53
54     ## Conversational video-QA ? build the bookkeeping NOW, punt the model
55     **Punt the conversational *feature*** until extraction quality is solid (current LOVO
~0.583 ? "very high"; a chat UI
56     over a weak extractor exposes the weakness). **Reject end-to-end-VLM-answers-each-turn**
(expensive; bad at exact
57     counting; the strong teachers are license-forbidden). **But build the *seam* now** (the
brief requires it):
58     - **Architecture = "extract once, query many":** EXTRACTION (TAL head ? later VLM) emits
structured timestamped
59     spans ? a durable **per-video EVENT STORE** ? a **structured QUERY layer** (text-to-
SQL + ffmpeg clip-cut). The
60     cited queries ("longest rally", "all service faults", "rallies > 20 shots") are
**structured-numeric ? SQL, not a
61     VLM job** (SQL counts exactly where VLMs hallucinate).
62     - **Designed-now seam:** make the **per-video event-index schema a first-class data
contract** alongside the M0.1
63     corpus spine: per rally `{video_id(MD5), rally_ordinal, sport, start, end, shot_count,
ending_reason/fault_tags,
64     derived_clip_ref, provenance, confidence}`. **Reserve the event/fault-tag taxonomy

```

```

(e.g. `service_fault`) up front**
65     even if unpopulated. ? This is a real DB delta (a `PRAGMA user_version` migration):
`play_segments` today has
66     `video_id/start_time/end_time/duration/server/receiver/metadata` only; `highlights`
isn't a table yet ? so it's an
67     *extension with new columns + a highlights table*, not a no-op. **Punt** the NL
parser/agent until quality clears the bar.
68     - Paid Gemini may answer genuinely open-ended edge questions **at inference** over our
structured index (allowed ?
69     it reasons over our data, it is not trained on its outputs).
70
71     ## Multisport (single sport-agnostic model)
72     - **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned
across sports (sport = an input
73     feature / per-sport calibration). The collector + indexer are already sport-generic. ?
**ACTION: rename the repo off
74     "badminton" ASAP** (owner emphatic) ? consistent with this.
75     - **What multisport changes = DATA + DETECTORS (scale up, the accepted "good
problem"):** each new sport needs its own
76     labeled matches (split by match) + a **clean trajectory detector** to feed the head's
features. **The binding blocker
77     is per-sport detectors:** WASB (badminton) has the unresolved **C3** checkpoint-
provenance issue, and **clean MIT/
78     Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel are not yet
identified.** Sequence by public-data
79     richness: **badminton ? tennis/TT ? pickleball/padel.**
80     - ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not
proven ? our corpus is badminton-only,
81     and the evidence once cited for it (RacketVision) was *refuted on verification* (it's
ball-tracking, not temporal). The
82     concept is sound on first principles (rally signal = a racquet-sport invariant) but
must be validated once a 2nd-sport
83     corpus exists. Don't cite RacketVision as evidence.
84
85     ## Language ? Python everywhere; on-device runtime is a deferred, layer-local export
shell
86     - **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer);
offline/batch, no latency hot path.
87     - **Platform = no Python hot path to escape** ? the heavy bytes/GPU already run out-of-
process (ffmpeg, WASB-in-WSL, cloud
88     GPU per-job); the control plane is I/O-bound. **Reject a Go/Rust hot-path split** ?
single-digit-ms wins dwarfed by
89     ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval
harness + provider registry.
90     - **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate
our dev language:** train in
91     Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device
shell (Swift/Kotlin/C++/Rust)
92     runs the artifact with zero Python. **The one concrete constraint now: train with
export-clean ops** so the compression
93     target stays reachable without re-architecting. (On-device base stays on the
Qwen3-VL-4B Apache lineage ? *not* Gemma-3-270M.)
94     - Escalation order if a real Python CPU hot path ever appears: free-threading ?
numpy/vectorize ? Cython/PyO3 ? only then a
95     separate service. **None exists today.**
96
97     ---
98
99     ## Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)
100
101     ### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) .
*greenlight item*
102     Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index
schema (the conversational seam)
103     ? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end,

```

```

ending_reason/fault_tags,
104     shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
105     labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL
ActivityNet-JSON · VLM clip?QA),
106     one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector`
(system-of-record; extends PR #13).
107     **Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match
enforced; **no Gemini-sourced rows in any training view**.
108
109     ### M0.2 ? Grow the corpus · **DONE (n=6), running** · *owner action + my scoring loop*
110     6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-
match + LOVO recorded; the
111     "contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club
match ? margin. **Bottleneck =
112     data + calibration, not method.** Keep growing per new sport.
113
114     ### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) ·
*greenlight item*
115     - *(i) PURGE the Gemini-silver TRAINING path** ? the live no-paid-LLM-training
violation is
116     **`backend/eval/distill_local.py`** (it trains a classifier on Gemini-silver CSV
labels ? `output/local_rally_model.json`)
117     and the threshold distillation in `calibrate_local.py`. (NB:
`backend/eval/training.py::label_candidates` is a *pure,
118     source-agnostic* function ? there is no store by that name; the earlier
"training.label_candidates" pointer was wrong.)
119     Nothing Gemini-trained is in git (artifacts live under gitignored `output/`), but the
path that regenerates them is one
120     command away ? retarget to human + open-teacher (own-model/WASB) labels; Gemini =
eval/reference/fallback only.
121     - *(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint
provenance is a **commercial-ship
122     blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a
clean MIT/Apache detector swap.
123     ? **own-weights is an un-scoped NEW workstream, not ROADMAP Phase 4:** per ADR-002 our
temporal labels can't train the
124     shuttle detector (it needs per-frame coordinate labels we don't have). **Multiplies
per sport** (incl. any TT-on-WASB use).
125     Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).
126
127     ### M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) · *greenlight item*
128     Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over
**ActionFormer (MIT, 1:1
129     rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not*
the Commons-Clause repo),
130     reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item):
a FLOOR of "beat the honest n=6
131     heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU**
target for "very high P/R" (e.g.
132     F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory.
Trains on the GPU/WSL box.
133
134     ---
135
136     ## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
137     Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it
becomes the rally/highlight
138     extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no
overfitting to declare victory.
139
140     ## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus +
explicit go)
141     - **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a
promising cue here** ? our E1 probe

```

```

142      (PR #35) found it weak (~2.2× discrimination, AUC ~0.6) and the foreground-audio idea
also failed this session; keep it
143      parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet
first).
144      - **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a
temporal-IoU-vs-golden reward,
145      our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on
Modal (no-access) / RunPod-Secure.
146      - **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-
hosted VLM vs Gemini) ** a real
147      VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples
use 678 GPUs ? **budget is the
148      largest open risk and is currently unquantified**). Don't stand up an owned serving
stack until these clear.
149
150      ---
151
152      ## Decisions / status
153      | # | item | status |
154      |---|---|---|
155      | 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4
harness** | awaiting owner |
156      | 2 | WASB C3 ? first-class action item (own-weights / clean swap) | **owner: yes** ? |
157      | 3 | Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights) | **owner: yes** ?
|
158      | 4 | Collector = corpus + event-store system-of-record | **owner: confirm?** |
159      | 5 | Rename repo off "badminton" | **owner action** ? |
160      | 6 | Conversational-QA: build seam now, punt model | **rec accepted?** |
161
162      ## Open risks / unknowns (none block Phase-0 research)
163      Budget/cost to reach Gen-2 **unquantified** (largest risk) · ship-gate "very high P/R"
target undefined · per-sport clean
164      trajectory detectors not yet identified · base-model pretraining provenance unauditale
(risk-acceptance) · real VLM
165      VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-
train (minors-excluded) for cloud
166      training · on-device 4B latency on 8 GB unmeasured.
167
168      ---
169      *Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing
discipline holds: the committed
170      next platform slice remains the async job model (single-tenant); only the M0.1?M0.4
foundation is safe to start.*
171

```

6. assistant (2026-06-14T08:24:19.155Z)

The git pull failed (multiple branches issue), but per the global instructions I proceed read-only on local state. Let me read the fusion plan and explore the actual harness/eval/LOVO code that the personalization pivot would rely on.

7. user (2026-06-14T08:24:19.607Z)

```

1      # Multi-signal rally fusion ? add a human (person) cue alongside the shuttle
2
3      **Status:** **P1 DONE** (2026-06-13) ? the person cue is extracted to
4      `backend/pipeline/detectors/person_detector.py` (`PersonDetector`), consumed by
`yolo_hybrid` with no
5      behavior change. **P2?P4 remain owner-gated.** Phases below are sequenced but each is a
separate PR.
6      **Owner decisions captured** in [$9](#9-open-decisions-for-the-owner).
7
8      > **Companion docs:** the 2-layer mental model is
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md);
9      > the modular-fusion architecture this builds on is **ADR-007**

```

```

([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md));
10     > the permissive-dependency rule is **ADR-002 / ADR-005**; the audio cue (the #1 next
cue, already
11     > scaffolded) is [archives/research/AUDIO_RALLY_DETECTION_PLAN.md](archives/research/AUD
IO_RALLY_DETECTION_PLAN.md).
12     > The vendor labels that gate the learned path come from
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md)
13     > and the labeling spec in [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md).
14     > **How each cue here is added/tested without production regression** ? A/B, shadow
eval, and the
15     > promote-only-on-lift discipline ? is [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md)
(every P1?P4 cue
16     > below lands flag-gated, default OFF, promoted only on measured LOVO lift).
17
18     ---
19
20     ## 1. Context ? why this, why now
21
22     Rally detection today runs **one cue at a time**. Each segmenter independently turns a
single signal
23     into rally windows:
24
25     | Segmenter | Signal | File |
26     |---|---|---|
27     | `wasb_hybrid` / `tracknet_hybrid` | shuttle velocity (frozen WASB/TrackNet trajectory)
| `backend/pipeline/detectors/tracknet_runner.py` |
28     | `yolo_hybrid` (the current `default_segmenter`) | person velocity (torchvision +
ByteTrack) | `backend/pipeline/segmenters/yolo_hybrid.py` |
29     | `motion` | raw pixel motion | `backend/pipeline/segmenters/motion.py` |
30     | `gemini` | a cloud VLM watching the whole clip | `backend/providers/gemini.py` |
31
32     **There is no fusion.** The owner's directive: make the detector **use the shuttle *and*
the humans
33     together** ? and be ready to fold in more signals (audio, later pose) ? delivered
34     **MIT / Apache-2.0 / BSD-clean** so we can ramp commercially without a licensing
rethink.
35
36     This is **not greenfield** ? it operationalizes work the repo already blessed:
37
38     - **`HOW_RALLY_DETECTION_WORKS.md` open question:** **"Fusion: combine shuttle-motion
(WASB) with
39     player-motion for static cams."
40     - **ADR-007** already designed a **modular "rally layer"** that fuses cues **outside* the
frozen WASB
41     model, with **audio as the #1 next cue and person/pose parked as #2**. This plan picks
up the parked
42     person cue and generalizes the fusion seam so future cues are "register a producer,"
not "re-touch
43     the segmenter."
44     - **`backend/pipeline/detectors/rally_gate.py` already exists** ? the net-crossing gate,
explicitly
45     labelled in its own docstring as **"Gate A in the over-fire fix spectrum (**B = player-
stance state
46     machine, future**)." The person cue in this plan **is that named-but-unbuilt Gate
B.**
47
48     ### The owner's "held-shuttle" false positive ? what's actually going on
49
50     The owner observed a "rally" that was just a player **holding / flicking the shuttle in
hand** between
51     points. This is the **exact* failure `rally_gate.py` was written to kill:
52
53     > **"WASB over-detects: when a player flicks/tosses the shuttle back to the opponent for
service after
54     > losing a point, that single trajectory looks like a rally. The discriminator is

```



```

structural: a real
55     > rally has the shuttle crossing the net MULTIPLE times; a single service feed crosses
~once."*
56
57     **The structural fix already exists and is validated offline ? but it is NOT wired into
the production
58     runtime path.** `rally_gate.apply_gate(..., min_crossings=2)` is called only from the
**eval drivers**
59     (`distill_local.py`, `eval_partial_labels.py`, `export_wasb_segments.py`,
`calibrate_local.py`); there
60     is **no `min_crossings` knob in `IndexingConfig`** and the live segmenters don't apply
it. So the
61     single cheapest, highest-confidence win in this whole plan is **P2's first step: fold
the existing
62     Gate A into production.** Everything else (the person cue / Gate B, the learned fusion)
is robustness
63     on top of that.
64
65     ### Why humans help ? and the honest risk
66
67     The beachhead videos are **static tripod** (Bangalore academies,
[PMF_MARKET_RESEARCH.md](PMF_MARKET_RESEARCH.md)),
68     so player presence/motion is a usable cue. ADR-001's warning was specifically about
**broadcast pan**
69     cameras, where player-motion fooled the old heuristic ? *that failure mode does not
apply to a fixed
70     court cam.* But we keep **the shuttle as the camera-invariant primary** so the general
bet stays robust
71     on other camera types. Humans then add:
72
73     - **Precision** ? reject shuttle-on-floor / warm-up knock-ups / held-shuttle feeds when
the court
74     isn't occupied by ?2 players in a play posture.
75     - **Recall** (via the learned classifier) ? features that help bridge shuttle
occlusion/blur gaps.
76
77     ---
78
79     ## 2. Core architecture ? one rally-fusion layer, cues are swappable producers
80
81     Formalize ADR-007's diagram. Every cue is a **black box that emits a time-aligned
signal** (per-frame
82     activity and/or per-window features); **fusion happens in *our* layer, never inside a
frozen model.**
83
84     ```
85     frames ?? WASB (frozen, MIT)      ?? shuttle trajectory ???
86     frames ?? person detector (ours)  ?? tracks + boxes ??????
87     audio  ?? hit detector (ours)     ?? hit events ???????????? RALLY LAYER (ours) ??
rallies
88     [frames ?? pose/stance (ours; parked #3) ?? stance ???????? feature + decision fusion
89     ```
90
91     - **Shuttle = primary, camera-invariant.** Windows still **seed** from
92     `TrackNetRunner.trajectory_to_action_windows()` (`tracknet_runner.py:234`). Other cues
*enrich and
93     adjudicate* shuttle-seeded windows; they do **not** replace the shuttle seed. Person
runs only over
94     **shuttle-seeded candidate windows + padding**, not the whole video ? which bounds the
added cost.
95     - **A small `RallyCue` contract.** Each producer yields `(frame_idx ? features)` + per-
window stats.
96     Adding a cue = register a producer; the existing `SegmenterRegistry`
(`segmenters/base.py:35`) is the
97     template for the pattern. Keep the single-cue segmenters registered as fallbacks.

```

```

98
99     ---
100
101     ## 3. The two fusion modes (both ? per the owner decision)
102
103     ### 3.1 Decision-level gate (safe baseline, ships first, no labels needed)
104
105     ```
106     rally_active = shuttle_motion ? (?2 persons in court zone ? recent person track)
107     ```
108
109     The `? recent-track` clause tolerates 1?2 frame person-detector misses so the AND-gate
    can't cause a
110     recall cliff. This composes with the existing net-crossing Gate A:
111
112     ```
113     keep window ? net_crossings ? 2                (Gate A ? EXISTS in rally_gate.py, eval-
    only today)
114             ? (?2 in-court players ? recent track)  (Gate B ? the person cue, NEW)
115     ```
116
117     Gate A kills the service-feed / held-shuttle case structurally; Gate B adds court-
    occupancy evidence
118     for the cases a single trajectory can still fool (e.g. a knock-up that does cross the
    net). No training
119     data required ? this is the label-free fusion that ships before any golden labels exist.
120
121     ### 3.2 Feature-level (primary, the learned path)
122
123     The cues emit a small set of aggregated, scale/translation-invariant per-window
    features ? NOT raw
124     coordinates. Raw pixel coordinates would memorize *this* court/camera and fail to
    generalize off a
125     still-small labeled set; counts / ratios / zone-membership / two-sidedness transfer.
126
127     What already feeds the classifier (`distill_local.py:40`, `FEATURE_NAMES`):
128     `duration, peak_velocity, mean_velocity, active_ratio, net_crossings` ? note
    net_crossings` is
129     already a feature (computed via `rally_gate.gate_windows`).
130
131     Person features to add (~4, deliberately few):
132
133     | Feature | Meaning | Rally signal |
134     |---|---|---|
135     | `players_in_court` | median count of persistent in-court tracks | ? 2 (singles) / 4
    (doubles); 0?1 = dead time |
136     | `two_sided_motion` | are moving players on both net-halves? | rally = two-sided;
    one person feeding = one-sided |
137     | `mean_player_motion` | aggregate normalized player velocity | rally = sustained;
    resting/standing = low |
138     | `in_court_ratio` | fraction of frames with ?2 in-court players | track stability vs
    flicker |
139     | `shuttle_player_colocation` | fraction of frames the shuttle sits inside/adjacent to a
    person box (hand region) | held = high; in-flight = mostly free space ? the direct held-
    shuttle discriminator |
140
141     These join the shuttle features in the candidate `stats` JSON
142     (`database.py::add_candidate_segment`) and the feature vector of the existing
    distilled classifier ?
143     the tiny numpy-only L2 logistic regression in `backend/eval/classifier.py` (ADR-004),
    NOT a new net.
144     It scores each candidate window `P(real rally)` and replaces the Gemini call at runtime.
145
146     What the model learns: the weights on those few features ? e.g. down-weight a
    shuttle-motion

```

```

147     window with 0?1 in-court players / one-sided motion / high shuttle-colocation (warm-up,
floor shuttle,
148     held-shuttle feed); up-weight ?2 players, two-sided, sustained motion, shuttle in free
flight.
149
150     **Small-N discipline (non-negotiable):**
151     - Keep to **~4 person features** ? more features on a still-small labeled set overfit;
that's exactly
152     why the model stays a logistic regression, not a net.
153     - Use the **leave-one-video-out (LOVO)** protocol that `distill_local.py` **already
implements**: train
154     on the other videos' candidates, predict the held-out *video*, score vs its labels ?
never a
155     held-out *window* from the same video (windows in one video are correlated; that
flatters the score).
156     - Training labels today are **Gemini silver labels** (distilled offline, ADR-004/007 ?
Gemini is a
157     teacher, never a runtime dependency, never a training-data *source* for owned
weights); **golden
158     labels are the honest measuring stick**, and as the Roora golden corpus grows they can
also become
159     training labels. Until the labeled set is larger, treat the learned weights as
160     **calibration / direction-check, not "a fully general model"*** ? the decision-level
gate ($3.1) is
161     the shippable, label-free fusion in the meantime.
162
163     > *(Gated on golden labels existing ? the same blocker ADR-007 flags for audio. See
164     > [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md).)*
165
166     ---
167
168     ## 4. Shuttle-state: in-flight vs in-hand (the misclassification, mapped to real code)
169
170     The "shuttle moved" seed conflates a shuttle **in flight** with a shuttle **carried in
hand**. The
171     discriminators, and where each lives:
172
173     | Signal | In-flight | Held / static | Status |
174     |---|---|---|---|
175     | `net_crossings` | ? 1 (usually several) | ~0 (held) / ~1 (single feed) | **EXISTS**
(`rally_gate.py`); eval-only ? wire to prod |
176     | `shuttle_player_colocation` | low (free space) | high (near a hand) | **NEW** (needs
the person cue) |
177     | `direction_reversals` | several (racket contacts) | ~0 | derivable from trajectory;
partial overlap with crossings |
178     | `peak shuttle speed` | high | walking speed | derivable from existing velocity |
179
180     So the in-flight-vs-held fix is **mostly Gate A (already built) + the new colocation
feature from the
181     person cue**. **No new label column is needed** ? the golden labels already start a
rally at the
182     *serve-contact frame* and treat all pre-serve / held-shuttle time as dead time (see
183     [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md)), so held-shuttle moments
are negatives
184     by construction and are the ground truth that proves the detector wrong.
185
186     ---
187
188     ## 5. Adversarial mitigations (integrating safely)
189
190     - **Court-region mask** ? polygon around the court; drop persons outside it (spectators,
coaches,
191     adjacent courts). Config-supplied per camera; ~1 ms. Source: a manual per-camera
polygon first;
192     the **4 court corners + net-line** that Roora labels once per video (see the labeling

```

```

doc) build
193     this mask **and** the net split that `two_sided_motion` / `net_crossings` need.
194     - **Player-count constraint** ? expect **2 (singles) / 4 (doubles)** persistent in-court
tracks;
195     suppress transient/extra detections. The `singles_or_doubles` manifest flag sets the
expectation.
196     - **Temporal association** ? reuse `supervision` **ByteTrack** (already in
`yolo_hybrid`); reject
197     singleton detections; smooth across frames.
198     - **Multi-court rejection** ? proximity to the primary court region (homography
optional, later).
199     - **Domain shift (COCO frontal ? side/overhead court)** ? start **box-only** (robust);
keep the
200     detector swappable. Fine-tuning a person detector is **parked** ? we have no in-house
person labels,
201     and the **paid-LLM/training guardrail + minors-exclusion** apply to any future person-
training data.
202     - **Fusion-degradation guard** ? shuttle stays primary; person is gate + feature, never
the sole
203     trigger; the single-cue segmenters remain registered fallbacks.
204
205     ---
206
207     ## 6. Box vs pose
208
209     **Box-first.** Occupancy + count + two-sided motion + shuttle-colocation is enough for
both the gate
210     and the features, and box detection is the cheap, robust win. **2D pose is parked behind
the same
211     `RallyCue` interface** ? pose (MediaPipe Pose / RTMPose, both Apache-2.0) buys serve-
ready stance
212     gating and shot cues later, at the cost of a second model. This matches ADR-007 parking
pose as the
213     #2/#3 cue.
214
215     ---
216
217     ## 7. Performance
218
219     The person detector is the **existing in-process torchvision model** run at **frame-skip
375**
220     (`yolo_frame_skip` default 3) with track interpolation between sampled frames. The
shuttle keeps
221     running in its separate WSL process ? the two are **separate processes**, so a "shared
backbone" isn't
222     free; budget realistically as the existing `yolo_hybrid` cost minus skip savings. Person
runs **only
223     over shuttle-seeded windows + `ai_handoff_padding`**, not the whole video, which bounds
the cost.
224
225     ---
226
227     ## 8. Licensing (the guardrail table ? all green)
228
229     Every dependency stays **MIT / Apache-2.0 / BSD** (ADR-002 / ADR-005; `ultralitics` was
dropped for
230     AGPL-3.0).
231
232     | Cue / model | Code | Weights / data | Verdict |
233     |---|---|---|---|
234     | Shuttle: WASB-SBDT, TrackNetV3 | MIT | frozen badminton weights | ? (ADR-001/002) |
235     | Person (default): torchvision Faster R-CNN / RetinaNet / FCOS | BSD/MIT | COCO-
pretrained | ? already in repo (`_DETECTOR_FACTORIES`) |
236     | Person (escalation): **YOLOX**, **RT-DETR / RT-DETRv2** | Apache-2.0 | COCO /
Objects365 | ? |

```

```

237 | Pose (parked): **MediaPipe Pose**, **RTMPose / MMPose** | Apache-2.0 | COCO-structure
| ? |
238 | Tracking: `supervision` ByteTrack | MIT | n/a | ? already in repo |
239 | **Banned** | **Ultralytics YOLOv5/8/11 (AGPL-3.0)** . **MonoTrack (Adobe, non-comm)**
. **YOLO-NAS (Deci, non-comm)** | ? | ? |
240
241 **COCO caveat to record:** COCO *annotations* are **CC BY 4.0** (commercial OK *with
attribution*);
242 COCO *images* are Flickr-ToS (per-image). Pretrained-**weight** use is clean; record the
attribution.
243
244 ---
245
246 ## 9. Open decisions (for the owner)
247
248 1. **Court-region mask source** ? manual per-camera polygon (simple, ship first) vs auto
court
249 homography/line detection (later). **Recommend manual first**, fed by Roora's court-
corner labels.
250 2. **Person detector default** ? keep torchvision (zero new deps, BSD/MIT) vs adopt
YOLOX/RT-DETR
251 (Apache, faster) now. **Recommend torchvision first**, escalate only if eval shows a
speed/accuracy gap.
252 3. **Pose now or parked** ? **recommend parked** behind the interface (box-first).
253 4. **Golden labels** ? feature-level fusion (P3) is **blocked on golden rally labels**;
the source is
254 the VLC `rally-annotator` + the Roora vendor pilot
([GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md)).
255
256 ---
257
258 ## 10. Phased execution (owner-gated; ONE PR per slice, off `master`)
259
260 - **P0 ? this design doc** +
[ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md), indexed in
261 `docs/README.md`, cross-linked from `HOW_RALLY_DETECTION_WORKS.md` and ADR-007. *(this
deliverable;
262 docs-only, no code.)*
263 - **P1 ? extract the shared person cue:** ? **DONE (2026-06-13).** Lifted the
torchvision detector +
264 ByteTrack + velocity out of `yolo_hybrid` into
`backend/pipeline/detectors/person_detector.py`
265 (`PersonDetector`); `yolo_hybrid` now orchestrates only and delegates to `self.person`
(no behavior
266 change ? regression-safe, proven by the unchanged process_video output tests). New
267 `tests/test_person_detector.py`; suite +1 green; person_detector is type-clean (left
mypy quarantine
268 for the moved code). The fusion segmenter (P2) reuses this same producer over shuttle-
seeded windows.
269 - **P2 ? the FusionSegmenter (Gate A + Gate B + person-feature persistence):** ? **DONE
(2026-06-14).**
270 Shipped `backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (registry
`fusion_hybrid`,
271 **default-OFF** ? `default_segmenter` unchanged), subclassing the trajectory engine
via two identity
272 hooks (`_enrich_candidate_stats`, `_select_for_handoff` ? wasb/tracknet stay byte-
identical, adversarially
273 verified). It seeds windows from the shuttle substrate, **always persists
`net_crossings` (Gate A signal),
274 and ? when `indexing.fusion.cue_flags.person` is on ? runs the person cue over each
window's frames
275 (`PersonDetector.detections_per_frame`) and persists the ~5 person features
(`fusion_features.py`,
276 pure + unit-tested). Served gating is optional + default-OFF: `fusion.min_crossings`
(Gate A) /

```

```

277     `fusion.min_players` (Gate B) skip sub-threshold windows from the handoff while
**persisting the full
278     superset** (offline substrate intact). Court mask = optional `fusion.court_polygon`
(None ? count-only).
279     Lift to be measured via the harness; held-shuttle-drop + person-feature values are
GPU/owner-side.
280     - **P3 ? feature-level (learned) fusion:** add the person features (incl.
`shuttle_player_colocation`)
281     to `distill_local` / `classifier`; recalibrate IoU on the golden set; report **LOVO
lift on held-out
282     videos**. *(Gated on golden labels.)*
283     - **P4 ? additional cues:** wire **audio** (ADR-007's #1, already scaffolded in
284     `backend/pipeline/audio/hit_detector.py`) and later **pose**, through the same
`RallyCue` interface.
285     Each is add-a-producer, not a refactor.
286
287     ---
288
289     ## 11. Verification
290
291     - **Offline, runnable in this dev container (no GPU/torch/cv2):** unit tests for the
fusion gate logic
292     and `RallyCue` aggregation (court-mask filter, player-count constraint, `? recent-
track` tolerance)
293     and the shuttle-state signals (net-crossing count ? extends the existing `rally_gate`
tests ? and
294     person-colocation) with **synthetic trajectories + person boxes**. A key-license
assertion test
295     (no `ultralytics` import; detector factories stay torchvision/Apache).
296     - **Machine-side (owner, GPU/WSL ? the full pipeline can't run in this container):** run
the
297     `FusionSegmenter` on a real static-tripod clip; confirm **held-shuttle / warm-up /
spectator false
298     positives drop** vs shuttle-only (this should reproduce the owner's observed bug and
show it fixed ?
299     start by confirming Gate-A-in-production alone fixes it); **LOVO IoU lift on the
golden set** for the
300     learned path.
301     - **Docs (P0):** links resolve; both docs indexed in `docs/README.md`; this doc cross-
linked from
302     `HOW_RALLY_DETECTION_WORKS.md` (the fusion open-question) and ADR-007.
303

```

8. user (2026-06-14T08:24:20.125Z)

```

1     # Experiment harness ? A/B + shadow eval for rally-detection quality (no-regression by
design)
2
3     **Status:** **P1 IMPLEMENTED** (`backend/eval/experiment.py` +
`tests/test_experiment.py`, shipped
4     2026-06-13) ? the `Variant` layer, `compare()` (labeled A/B, LOVO-honest),
`compare_unlabeled()`
5     (SxS on NEW unlabeled footage), and the leaderboard. **P2?P4 remain owner-gated.**
Phases below are
6     sequenced but each is a separate PR.
7
8     > **Companion docs:** the cues this experiments *on* are in
9     > [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md); the scoring mechanics are
10    > [EVALUATION.md](EVALUATION.md); the no-regression gate + golden labels +
`baseline.json` come from
11    > [GOLDEN_SET_VENDORIZING.md](GOLDEN_SET_VENDORIZING.md); the modular-cue architecture is
**ADR-007**
12    > ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md)); the distilled
classifier is
13    > **ADR-004**. The 2-layer mental model is

```

```
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md).
14
15     ---
16
17     ## 1. Context ? why this, why now
18
19     PR #112 (merged) added the multi-signal fusion design ? shuttle + person + audio cues,
all **default
20     OFF**. The owner's directive for *how* we evolve quality from here:
21
22     > "Evolve the codebase so we can keep finding new ways to improve quality and
experiment with them ?
23     > or add them as a signal ? without suffering any regressions if the idea doesn't work
out. Rolling
24     > back the code would be too dangerous."*
25
26     So **experimentation and deployment must be decoupled.** The contract:
27
28     1. Merge experimental code freely ? it's **gated OFF**, so it cannot change production
behavior.
29     2. Test it **offline against golden labels** (most experiments never touch the served
path at all).
30     3. Promote a variant to default **only on measured lift**, through a CI eval gate.
31     4. **"Rollback" = flip a config flag, never `git revert`.** The proven path is a pinned
variant that is
32         always present.
33
34     **The pleasant surprise (Explore audit, 2026-06-13):** ~80% of this harness already
exists and is
35     simply not composed *as a system*. The missing piece is a thin **variant/experiment
layer + the
36     discipline**, not a platform. §4 is the exact code surface so the next session does not
re-discover it.
37
38     ---
39
40     ## 2. The thesis ? separate expensive DETECTION from cheap DECISION
41
42     The expensive, GPU/WSL-bound, risky part is **detection**: "what signals exist per
frame" (shuttle
43     WASB trajectory, person tracks/boxes, audio hits). The cheap, swappable, pure-Python
part is the
44     **decision**: "given those signals, where are the rallies" (windowing + gates + the
classifier).
45
46     > **Run detection once per video, persist all per-cue signals + per-candidate features,
then re-score
47     > any number of variants OFFLINE ? deterministically, with no GPU and zero production
risk.**
48
49     This is not aspirational ? `distill_local.py` and `calibrate_local.py` already do
exactly this on
50     stored WASB trajectories. The candidate-segments table (`stats` JSON) is the persistence
substrate.
51     ? Most experiments are pure re-scoring of stored data; they never enter the served
pipeline.
52
53     ---
54
55     ## 3. What already exists (the audit ? do NOT rebuild)
56
57     Exact public surface, with `file:line`. **This table is the anti-duplicate-work
artifact** ? start here.
58
59     ### 3.1 Scoring (variant-agnostic ? grades any `List[Interval]`)
```

```

60     `backend/eval/metrics.py`
61     - `interval_iou(a, b) -> float` (:15) ? temporal IoU of two `(start,end)` intervals.
62     - `match_intervals(preds, gts, iou_threshold=0.5) -> MatchResult` (:48) ? greedy 1-to-1
IoU matching.
63     - `score(preds, gts, iou_threshold=0.5, match_result=None) -> Score` (:104) ? P/R/F1,
mean IoU, boundary errors; `Score.as_dict()``.
64     - `pr_curve(preds, gts, thresholds=None) -> List[Score]` (:138).
65
66     ### 3.2 A/B + cross-validation primitives (the comparison engine already exists)
67     `backend/eval/calibration.py`
68     - `improvement_report(predict_fn, baseline_config, tuned_config, gts, iou_threshold=0.5)
-> dict` (:148) ? **before/after metrics + %?. This IS the A/B delta.**
69     - `leave_one_video_out(videos, configs, metric="f1", iou_threshold=0.5) -> dict` (:113)
? **LOVO: pick on other videos, score on held-out video (honest cross-video).**
70     - `cross_validate(predict_fn, configs, gts, k=5, metric="recall", iou_threshold=0.5) ->
dict` (:72) ? within-video k-fold overfit guard.
71     - `select_best(predict_fn, configs, gts, metric="f1", iou_threshold=0.5) -> (Config,
float)` (:61).
72     - `evaluate(preds, gts, iou_threshold=0.5) -> Score` (:41); `kfold_indices(n, k)` (:46).
73     - LOVO video dict shape: `{ "name": str, "predict_fn": PredictFn, "gts": [Interval] }`.
74
75     ### 3.3 No-regression gate + baselines (the promotion gate already exists)
76     `backend/eval/regression.py`
77     - `regress(db, video_id, ground_truth, stage="final", iou_threshold=0.5, detector=None,
tolerance=DEFAULT_TOLERANCE, baseline_path=DEFAULT_BASELINE_PATH) -> RegressionResult` (:104) ?
score stored preds vs GT, flag if P **or** R drops > tolerance.
78     - `regress_with_provider(...)` (:160) ? same, fetching GT via the annotation provider.
79     - `save_baseline(video_id, stage, precision, recall, f1, ..., gt_fingerprint=None)`
(:80) . `load_baselines(path) -> dict` (:68).
80     - `RegressionResult` (:43): `regressed`, `reasons`, `gt_hash`, baseline P/R,
`tolerance`, `has_baseline`; `as_dict()``.
81     - Default baseline file: `eval_baselines/regression_baseline.json` (:33).
82
83     `run_regression.py` ? CI-gatable CLI, `main(argv)` (:52). **Exit codes: 0 ok / 1
regression / 2 no-DB / 3 no-baseline.** Flags: `--video-id --tier --annotations
--stage{candidates,final} --detector --iou --tolerance --baseline --update-baseline --allow-
missing-baseline --json`.
84
85     ### 3.4 Pinnable model artifact
86     `backend/eval/classifier.py` ? `LogisticRegression` (pure numpy, L2):
`fit(X,y,feature_names=,class_weight="balanced")` (:40), `predict_proba` (:75),
`predict(threshold=0.5)` (:81), `to_dict`/`from_dict` (:86/:97), `save`/`load` (JSON)
(:107/:111). **A trained model = one JSON file ? a pinnable, hashable variant artifact.**
87
88     ### 3.5 Offline re-scoring substrate (persist once, re-score forever)
89     `backend/infrastructure/database.py`
90     - `add_candidate_segment(video_id, detector, start_time, end_time, chunk_index=None,
confidence=None, stats=None, detection_params=None) -> Optional[int]` (:240) ?
`stats`/`detection_params` stored as JSON; `candidate_segments` table cols incl. `stats`,
`detection_params`, `human_label`, `confirmed_segment_id` (schema :97).
91     - `query_candidate_segments(video_id, detector=None, only_unlabeled=False) ->
List[dict]` (:277) ? returns rows with deserialized `stats`.
92
93     ### 3.6 Label assignment + feature glue
94     `backend/eval/training.py`
95     - `label_candidates(candidate_intervals, gt_intervals, iou_threshold=0.5) -> List[int]`
(:50) ? y-labels by IoU match to golden (same matcher as the evaluator).
96     - `build_training_set(candidate_rows, gt_intervals, iou_threshold=0.5,
feature_fn=extract_yolo_features) -> (X, y, intervals)` (:64) ? pluggable `feature_fn`.
97
98     ### 3.7 Existing LOVO drivers to mirror (not duplicate)
99     - `backend/eval/distill_local.py` ? `run(specs, iou=0.5, threshold=0.5, model_out=None)`
(:98), CLI `python -m backend.eval.distill_local --manifest videos.json --model-out ...`;
`FEATURE_NAMES = [duration, peak_velocity, mean_velocity, active_ratio, net_crossings]` (:40);
already does LOVO over Gemini silver labels.

```



```

100     - `backend/eval/calibrate_local.py` ? `run(specs, iou=0.5, metric="f1")` (:77);
`LocalConfig = (velocity_thresh, merge_gap, min_window_duration, min_crossings)`; grid + LOVO.
101     - `backend/eval/calibrate_wasb.py` ? `run(...)` , `load_golden_gts()`;
`backend/eval/gt_loader.py::load_gt_intervals(csv_path, *, strict=True)` is THE canonical golden
reader.
102
103     ### 3.8 Registries + config knobs (variant selection)
104     - `backend/pipeline/segmenters/base.py` ? `SegmenterRegistry` (:35), singleton
`segmenter_registry` (:63); registered: `motion`, `gemini`, `yolo_hybrid`, `tracknet_hybrid`,
`wasb_hybrid`.
105     - `backend/annotations/base.py` ? `AnnotationProviderRegistry`, singleton
`annotation_registry`; providers `file`, `auto`.
106     - `backend/config/models.py::IndexingConfig` ? `default_segmenter` (:65, default
`"yolo_hybrid"`), `detector_model` (:72), `detector_score_threshold` (:73),
`yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75), `min_segment_duration` (:68),
`dead_time_merge_gap` (:69), `motion_threshold` (:67). **No `min_crossings` knob yet** (see
fusion P2).
107
108     ### 3.9 Confirmed ABSENT (no duplication risk)
109     Explore found **no** existing experiment / variant / shadow / A-B *machinery* ? only a
stray "A/B test"
110     aspiration comment in `backend/pipeline/detectors/base.py` and an "experiment E1" note
in
111     `AUDIO_RALLY_DETECTION_PLAN.md`. The `regress()` + baseline path is the canonical
comparison mechanism.
112
113     ---
114
115     ## 4. The variant abstraction (the one thin thing to add)
116
117     A **`Variant` = a declarative recipe**, not a code branch:
118
119     ```
120     Variant = (segmenter_name, IndexingConfig overrides, enabled cues + per-cue params,
classifier model path/hash)
121     ```
122
123     JSON-serializable; selected via the existing `segmenter_registry` +
`IndexingConfig.default_segmenter`.
124     **Control** = a *pinned, frozen* variant (recipe + model hash) that is always registered
and is the
125     default. Adding a new cue = a new Variant differing by one flag ? the control recipe is
untouched.
126
127     ---
128
129     ## 5. Three experiment modes
130
131     ### 5.1 Offline A/B (primary, label-driven, zero production risk)
132     `compare(videos, variant_A, variant_B)` ? a thin driver composing **existing**
functions:
133     `query_candidate_segments()` (load persisted features) ? re-score each variant ?
`calibration.improvement_report()`
134     + `calibration.leave_one_video_out()` + `metrics.score()`, graded vs golden
(`gt_loader.load_gt_intervals`).
135     Reports per-video **and** LOVO-honest aggregate IoU/P/R/F1 deltas. **This is the A/B
test, and it needs
136     almost no new scoring code** ? only the glue.
137
138     **Honest reporting (default ON, additive):** `compare(..., honest_stats=True)` also
attaches a **`honest`**
139     block ? per-metric bootstrap **CIs**, a paired **F1 CI + exact sign test** (C1), and
**F1@k +
140     segment-count ratio** (C4) ? *alongside* the bare-mean fields (existing output
unchanged; `honest_stats=False`

```

```

141     restores the legacy shape). A bare LOVO mean at n?6 is not promotable on its own; the
142     lift is real when the
143     ?F1 CI clears 0 **and** the sign test agrees. Wiring of the course corrections in
144     [ANALYZERS_AND_RECONCILERS.md](ANALYZERS_AND_RECONCILERS.md) §13/C1+C4 (helpers:
145     `backend/eval/eval_stats.py`,
146     `segmentation_metrics.py`); `record_experiment` carries the honest ?F1 into the
147     leaderboard.
148
149     ### 5.2 Shadow (for cues that touch detection, e.g. the person detector)
150     The new cue runs and **persists its signal** into the candidate
151     `stats`/`detection_params`, but the
152     *served* decision stays control. Offline you then ask "what would variant B have
153     scored?" via §5.1 ?
154     B never affects output until it wins. This is how a detection-touching change (not just
155     a re-scoring
156     change) earns its way in without risk.
157
158     ### 5.3 Promotion gate (no-regression, CI-enforced)
159     `regression.regress()` + `baseline.json` + `run_regression.py` exit codes. A variant
160     becomes the new
161     default **only if** it doesn't regress the golden set beyond `tolerance` (exit 1
162     blocks). Control is
163     pinned ? **rollback = flip `default_segmenter`/variant config, never a code revert.**
164
165     ---
166
167     ## 6. No-regression guarantees (the heart of the owner's ask)
168
169     - **New cues/signals default OFF** ? per-cue enable flag in `IndexingConfig`; merged
170     code can't change
171     behavior until explicitly enabled.
172     - **Control variant pinned** ? frozen recipe + classifier model hash, always registered
173     & default.
174     - **Promotion only via the eval gate** ? `run_regression.py` in CI (exit 1 blocks); no
175     silent default change.
176     - **Experiments are records** ? variant-spec hash ? per-video + LOVO scores + label-set
177     hash + timestamp,
178     persisted to a small JSON **experiment leaderboard** (generalizes `baseline.json`).
179     Answers "which
180     signals ever helped, on which video slices," and makes every result reproducible.
181     - **Decoupled events** ? "merge an experiment" and "change the default" are separate,
182     independently
183     revertible actions.
184
185     ---
186
187     ## 7. What to build later (gap list ? thin, additive, owner-gated)
188
189     1. ~**`backend/eval/experiment.py`** ? a `Variant` dataclass + `compare(videos, a, b)`
190     composing the §3
191     functions (no new scoring math).~ **DONE (2026-06-13).** Shipped `Variant` (+ pinned
192     `CONTROL`),
193     `compare` (labeled A/B with LOVO-honest learned-variant training, mirroring
194     `distill_local`),
195     `compare_unlabeled` (the **SxS-on-new-videos** mode ? agreed / A-only / B-only via
196     `match_intervals`,
197     no GT), `record_experiment` (? `eval_baselines/experiment_leaderboard.json`), and
198     `load_video_candidates`
199     (the `query_candidate_segments` bridge). JSON variant recipes first as recommended; a
200     `VariantRegistry`
201     only if the count grows. 19 tests incl. the no-regression invariant.
202     2. **Persist per-cue features** into candidate `stats` (person: `players_in_court`,
203     `two_sided_motion`,
204     `in_court_ratio`, `shuttle_player_colocation`; shuttle-state: `net_crossings`,
205     `direction_reversals`;

```

```

184         audio: hit times) so any future variant re-scores offline ? ties to
`MULTI_SIGNAL_FUSION_PLAN.md` §3.2/§4.
185     3. Per-cue enable flags in `IndexingConfig` (default `False`); `default_segmenter`
already exists.
186         Add the `min_crossings` knob (fusion P2) here too.
187     4. CI promotion lane using `run_regression.py` against the golden `baseline.json`;
write the
188         experiment-leaderboard JSON.
189
190     ---
191
192     ## 8. Phasing (owner-gated; ONE PR per slice, off `master`)
193
194     - P0 ? this doc, indexed in `docs/README.md`, cross-linked from
`MULTI_SIGNAL_FUSION_PLAN.md` and
195         `GOLDEN_SET_VENDURING.md`. *(this deliverable; docs-only, no code.)*
196     - P1 ? `experiment.py` `compare()` ? DONE (2026-06-13): thin wrapper over
`calibration` +
197         `metrics` + `query_candidate_segments`; pure-logic, fully tested in this dev container
(`compare`,
198         `compare_unlabeled`, `record_experiment`, the no-regression invariant).
`compare_unlabeled` adds the
199         SxS read on new unlabeled footage the owner asked for (run control + candidate on
a fresh video,
200         diff where they agree/diverge ? two reels via the existing `VideoCompiler` is the GPU-
side follow-on).
201     - P2 ? per-cue enable flags + feature persistence into candidate `stats` (default
OFF) ? couples
202         with fusion P1/P2.
203     - P3 ? CI promotion gate: `run_regression.py` lane on the golden `baseline.json`;
experiment-leaderboard JSON.
204
205     ---
206
207     ## 9. Scope guardrail
208
209     Single-tenant / local-first (per `PLATFORM_ARCHITECTURE.md`): a JSON experiment
registry + the
210     existing harness ? explicitly NOT MLflow / W&B / a service. Keep it the size of
`baseline.json`.
211
212     ---
213
214     ## 10. Verification
215
216     - Offline, runnable in this dev container (no GPU/torch needed for the glue): unit
test that
217         `compare()` is deterministic on synthetic candidate rows; a no-regression invariant
test asserting
218         "all new-cue flags OFF ? predictions byte-identical to control"; a CI-gate test
asserting
219         `run_regression.py` exit codes (0 ok / 1 regress / 3 no-baseline).
220     - Machine-side (owner, GPU/WSL): persist candidates once on a real clip, run
`compare(control,
221         fusion)` LOVO, confirm the reported lift matches a manual `metrics.score()` spot-
check.
222     - Docs (P0): links resolve; indexed in `docs/README.md`; cross-linked from
223         `MULTI_SIGNAL_FUSION_PLAN.md` and `GOLDEN_SET_VENDURING.md`.
224     </content>
225

```

9. assistant (2026-06-14T08:24:23.095Z)

Now let me look at the actual code for the LOVO/calibration, the classifier, and the n=6 statistics problem, plus the per-video calibration mechanics that the personalization pivot

leans on.

10. user (2026-06-14T08:24:24.064Z)

```
1      """Quantify rally-segmentation improvement ? and guard against overfitting.
2
3      This is the measurement layer for per-video / per-corpus calibration. It answers
4      two questions honestly:
5
6      1. """Did precision improve, by how much?""" ? `improvement_report` scores a
7          baseline config vs a tuned config on the same ground truth and reports the
8          before/after metrics and the % delta.
9
10     2. """Is that gain real or overfit?""" ? calibration must be measured on data the
11         tuning never saw, otherwise you're just memorising. Two protocols:
12         - `cross_validate` ? k-fold over one video's rallies: pick the config on the
13           train folds, score on the held-out fold. The `generalization_gap`
14           (train ? test) is the overfit alarm.
15         - `leave_one_video_out` ? the protocol that matters as more videos arrive:
16           pick the config on all *other* videos, score on the held-out video. Reports
17           per-video held-out scores + mean±std. This is the number to trust.
18
19     Everything is segmenter-agnostic: callers pass a `predict_fn(config) ->
20     List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same
21     harness works for WASB, yolo_hybrid, or anything else. Pure + unit-tested; no I/O.
22     """
23     from __future__ import annotations
24
25     import statistics as _st
26     from typing import Callable, Dict, List, Sequence, Tuple, Any
27
28     from backend.eval.metrics import score, Score, Interval
29
30     Config = Any                                # opaque to this module (e.g. a thresholds
dict)
31     PredictFn = Callable[[Config], List[Interval]]
32
33
34     def pct_improvement(before: float, after: float) -> float:
35         """Percentage change from `before` to `after` (e.g. 0.6 -> 0.75 == +25.0)."""
36         if before <= 0:
37             return float("inf") if after > 0 else 0.0
38         return (after - before) / before * 100.0
39
40
41     def evaluate(preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5) ->
Score:
42         """Precision / recall / F1 / mean-IoU of predicted vs ground-truth intervals."""
43         return score(preds, gts, iou_threshold)
44
45
46     def kfold_indices(n: int, k: int) -> List[Tuple[List[int], List[int]]]:
47         """Deterministic k-fold split of indices 0..n-1 into (train_idx, test_idx)."""
48         if n <= 0:
49             return []
50         k = max(2, min(k, n))                    # need >=2 folds; cap at n
51         folds = [list(range(i, n, k)) for i in range(k)]    # round-robin ? balanced,
deterministic
52         splits = []
53         for f in range(k):
54             test = folds[f]
55             train = [i for i in range(n) if i not in set(test)]
56             if test and train:
57                 splits.append((train, test))
58         return splits
```

```

59
60
61 def select_best(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
62                 metric: str = "f1", iou_threshold: float = 0.5) -> Tuple[Config, float]:
63     """Pick the config whose predictions maximise `metric` against `gts`."""
64     best_cfg, best_val = None, -1.0
65     for cfg in configs:
66         val = getattr(score(predict_fn(cfg), gts, iou_threshold), metric)
67         if val > best_val:
68             best_cfg, best_val = cfg, val
69     return best_cfg, best_val
70
71
72 def cross_validate(predict_fn: PredictFn, configs: Sequence[Config], gts:
List[Interval],
73                   k: int = 5, metric: str = "recall", iou_threshold: float = 0.5) ->
Dict[str, Any]:
74     """k-fold threshold calibration on ONE video's rallies (within-video overfit guard).
75
76     Per fold: choose the best config on the train rallies, score it on the held-out
77     rallies. A large `generalization_gap` (train >> test) means the thresholds are
78     overfitting that video's quirks.
79
80     Default metric is recall on purpose: predictions are global (whole-video
81     windows), so scoring them against a fold subset of GT would unfairly count the
82     other rallies' windows as false positives ? precision/F1 are not well-defined
83     per-fold. Recall ("did the tuned thresholds still detect the held-out rallies?")
84     is the honest within-video generalization signal. For precision/F1, use
85     `leave_one_video_out` (each video scored whole) or `improvement_report` on a
86     held-out video.
87     """
88     splits = kfold_indices(len(gts), k)
89     if not splits:
90         return {"error": "not enough rallies for cross-validation", "n": len(gts)}
91     train_scores, test_scores, picked = [], [], []
92     for train_idx, test_idx in splits:
93         gts_tr = [gts[i] for i in train_idx]
94         gts_te = [gts[i] for i in test_idx]
95         cfg, tr = select_best(predict_fn, configs, gts_tr, metric, iou_threshold)
96         te = getattr(score(predict_fn(cfg), gts_te, iou_threshold), metric)
97         train_scores.append(tr)
98         test_scores.append(te)
99         picked.append(cfg)
100     mean_test = _st.mean(test_scores)
101     mean_train = _st.mean(train_scores)
102     return {
103         "metric": metric,
104         "folds": len(splits),
105         "mean_test": mean_test,
106         "std_test": _st.pstdev(test_scores) if len(test_scores) > 1 else 0.0,
107         "mean_train": mean_train,
108         "generalization_gap": mean_train - mean_test,    # >~0.1 = likely overfitting
109         "picked_configs": picked,
110     }
111
112
113 def leave_one_video_out(videos: List[Dict[str, Any]], configs: Sequence[Config],
114                        metric: str = "f1", iou_threshold: float = 0.5) -> Dict[str,
Any]:
115     """Leave-one-video-out CV ? the honest metric as the corpus grows.
116
117     `videos`: list of {"name": str, "predict_fn": PredictFn, "gts": [Interval]}.
118     For each held-out video: pick the config that's best *pooled across the other
119     videos*, then score it on the held-out one. The mean held-out score is the
120     "precision on unseen videos" number; per-video spread shows stability.

```

```

121     """
122     if len(videos) < 2:
123         return {"error": "need >=2 videos for leave-one-video-out", "n": len(videos)}
124     per_video = []
125     for i, held in enumerate(videos):
126         others = [v for j, v in enumerate(videos) if j != i]
127
128         def pooled_score(cfg: Config, others: List[dict] = others) -> float:
129             vals = [getattr(score(v["predict_fn"])(cfg), v["gts"], iou_threshold),
metric)
130                         for v in others]
131             return _st.mean(vals)
132
133         best_cfg, best_train = max(((c, pooled_score(c)) for c in configs), key=lambda
t: t[1])
134         held_score = getattr(score(held["predict_fn"])(best_cfg), held["gts"],
iou_threshold), metric)
135         per_video.append({"video": held.get("name", f"video_{i}"),
136                         "held_out": held_score, "train_pooled": best_train, "config":
best_cfg})
137     held_scores = [p["held_out"] for p in per_video]
138     train_scores = [p["train_pooled"] for p in per_video]
139     return {
140         "metric": metric,
141         "mean_held_out": _st.mean(held_scores),
142         "std_held_out": _st.pstdev(held_scores) if len(held_scores) > 1 else 0.0,
143         "generalization_gap": _st.mean(train_scores) - _st.mean(held_scores),
144         "per_video": per_video,
145     }
146
147
148     def improvement_report(predict_fn: PredictFn, baseline_config: Config, tuned_config:
Config,
149                             gts: List[Interval], iou_threshold: float = 0.5) -> Dict[str,
Any]:
150         """The "+X% precision" headline: baseline vs tuned config on the same GT.
151
152         NB: for an *honest* headline, pass GT the tuning never saw (a held-out video),
153         or read the cross-validated `mean_test` from `cross_validate` instead.
154         """
155         b = score(predict_fn(baseline_config), gts, iou_threshold)
156         t = score(predict_fn(tuned_config), gts, iou_threshold)
157         return {
158             "baseline": {"precision": b.precision, "recall": b.recall, "f1": b.f1,
"mean_iou": b.mean_iou},
159             "tuned": {"precision": t.precision, "recall": t.recall, "f1": t.f1, "mean_iou":
t.mean_iou},
160             "precision_delta_pts": t.precision - b.precision,
161             "precision_delta_pct": pct_improvement(b.precision, t.precision),
162             "f1_delta_pct": pct_improvement(b.f1, t.f1),
163         }
164

```

11. user (2026-06-14T08:24:24.473Z)

```

1     """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3     The offline learned segmenter (ADR-004) must run locally with no external API and
4     no heavy ML stack ? the project already depends on numpy, and sklearn is not
5     installed. This implements just enough: standardized features, balanced-class
6     gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7     model is a small, human-readable, version-controllable file.
8
9     It deliberately stays simple: the rally-window features are few and roughly
10    linearly separable (high player motion + sustained activity => real rally), so a

```

```

11 well-regularized linear model is a sensible, inspectable baseline. Swap in a
12 richer model later behind the same fit/predict_proba/save/load interface.
13 """
14
15 import json
16 from typing import List, Optional
17
18 import numpy as np
19
20
21 class LogisticRegression:
22     def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23         self.lr = lr
24         self.epochs = epochs
25         self.l2 = l2
26         self.w: Optional[np.ndarray] = None
27         self.b: float = 0.0
28         self.mean: Optional[np.ndarray] = None
29         self.std: Optional[np.ndarray] = None
30         self.feature_names: Optional[List[str]] = None
31
32     @staticmethod
33     def _sigmoid(z: np.ndarray) -> np.ndarray:
34         # Clip for numerical stability (avoid overflow in exp).
35         return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37     def _standardize(self, X: np.ndarray) -> np.ndarray:
38         return (X - self.mean) / self.std
39
40     def fit(self, X, y, feature_names: Optional[List[str]] = None,
41            class_weight: str = "balanced") -> "LogisticRegression":
42         """Fit with optional balanced class weighting (rally windows are imbalanced
43         ? many spurious candidates per real rally)."""
44         X = np.asarray(X, dtype=float)
45         y = np.asarray(y, dtype=float)
46         if X.ndim != 2 or X.shape[0] == 0:
47             raise ValueError("X must be a non-empty 2D array")
48         self.feature_names = feature_names
49
50         self.mean = X.mean(axis=0)
51         self.std = X.std(axis=0)
52         self.std[self.std == 0] = 1.0 # guard constant features
53         Xs = self._standardize(X)
54         n, d = Xs.shape
55
56         if class_weight == "balanced":
57             n_pos = max(1, int(y.sum()))
58             n_neg = max(1, int((1 - y).sum()))
59             w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
60         else:
61             w_pos = w_neg = 1.0
62         sw = np.where(y == 1, w_pos, w_neg)
63
64         self.w = np.zeros(d)
65         self.b = 0.0
66         for _ in range(self.epochs):
67             p = self._sigmoid(Xs @ self.w + self.b)
68             err = (p - y) * sw
69             grad_w = Xs.T @ err / n + self.l2 * self.w
70             grad_b = float(err.mean())
71             self.w -= self.lr * grad_w
72             self.b -= self.lr * grad_b
73         return self
74
75     def predict_proba(self, X) -> np.ndarray:

```

```

76         if self.w is None:
77             raise RuntimeError("model is not fitted")
78         Xs = self._standardize(np.asarray(X, dtype=float))
79         return self._sigmoid(Xs @ self.w + self.b)
80
81     def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82         return (self.predict_proba(X) >= threshold).astype(int)
83
84     # --- persistence (small JSON, no pickle/joblib) -----
85
86     def to_dict(self) -> dict:
87         return {
88             "model": "logreg",
89             "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90             "w": self.w.tolist() if self.w is not None else None,
91             "b": self.b,
92             "mean": self.mean.tolist() if self.mean is not None else None,
93             "std": self.std.tolist() if self.std is not None else None,
94             "feature_names": self.feature_names,
95         }
96
97     @classmethod
98     def from_dict(cls, d: dict) -> "LogisticRegression":
99         m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100         m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101         m.b = float(d.get("b", 0.0))
102         m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103         m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104         m.feature_names = d.get("feature_names")
105         return m
106
107     def save(self, path: str) -> None:
108         with open(path, "w") as f:
109             json.dump(self.to_dict(), f, indent=2)
110
111     @classmethod
112     def load(cls, path: str) -> "LogisticRegression":
113         with open(path) as f:
114             return cls.from_dict(json.load(f))
115

```

12. assistant (2026-06-14T08:24:28.804Z)

Now let me look at the experiment.py harness, the honest-stats helpers (the n=6 problem), and the per-video calibration plus the correction loop that would serve as the per-user label source.

13. user (2026-06-14T08:24:30.221Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`           ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`           ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`         ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine`      ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`         ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`        ? `pct_improvement`
15

```



```

16 The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17 (per-frame signals ? run once on the GPU/WSL box, persisted into
18 `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19 signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20 given persisted candidate features it produces `List[Interval]` deterministically,
21 with no GPU and zero production risk. So most experiments are pure offline
22 re-scoring of stored data and never touch the served pipeline.
23
24 Three modes (`EXPERIMENT_HARNESS.md` §5):
25 - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26   aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27   held-out loop, but variant-parameterised.
28 - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29   variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30   human spot-checks (and what two highlight reels would show side by side).
31 - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32   (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Callable, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54 from backend.eval import eval_stats as _stats # C1: CIs + paired sign test
55 from backend.eval import segmentation_metrics as _segm # C4: F1@k + segment-count
56 ratio
57 scores
58 from backend.eval.score_calibration import fit_isotonic, fit_platt # C2: calibrate cue
59
60 logger = logging.getLogger(__name__)
61
62 # Where a comparison run appends one reproducible record. Tracked location (NOT under
63 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
64 # regression.DEFAULT_BASELINE_PATH.
65 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
66 _METRICS = ("precision", "recall", "f1", "mean_iou")
67
68 class ExperimentError(ValueError):
69     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70     score an unlabeled video with no pinned model, or a gate referencing an absent
71     feature)."""
72
73 # ----- Variant
74
75 @dataclass
76 class Variant:
77     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

```

```

79
80 A variant turns one video's persisted candidate windows + features into rally
81 intervals. Every knob defaults to the control behaviour (no gate, no learned
82 filter), so a variant that merely carries a new, disabled cue is byte-identical
83 to control ? the core no-regression guarantee.
84
85 Fields:
86 - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87   for the leaderboard and for selecting the served path (the existing
88   `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89   New cues are recorded here default-OFF.
90 - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91   ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
92   `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93   windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94 - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95   whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
96   cue persists that feature).
97 - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98   `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99   required for `compare_unlabeled` on a learned variant.
100 - ``feature_names`` ? the persisted features the learned filter consumes. When set
101   WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102   (honest) ? the recipe, not a pinned artifact.
103 - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104   overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105 """
106
107 name: str
108 segmenter: str = "wasb_hybrid"
109 config_overrides: Dict[str, Any] = field(default_factory=dict)
110 cue_flags: Dict[str, bool] = field(default_factory=dict)
111 min_crossings: int = 0
112 min_players: int = 0
113 classifier_model: Optional[str] = None
114 feature_names: Optional[List[str]] = None
115 threshold: float = 0.5
116 merge_gap: float = 1.0
117 # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118 # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119 # failure where a fixed 0.5 zeroed out a small-candidate video.
120 tune_threshold: bool = False # pick the F1-best threshold on the train fold
121 calibrate: Optional[str] = None # None | "platt" | "isotonic" ? calibrate
proba first
122
123 def is_learned(self) -> bool:
124     """True if this variant applies a learned classifier (pinned model or
recipe)."""
125     return self.classifier_model is not None or bool(self.feature_names)
126
127 def to_dict(self) -> dict:
128     return {
129         "name": self.name,
130         "segmenter": self.segmenter,
131         "config_overrides": self.config_overrides,
132         "cue_flags": self.cue_flags,
133         "min_crossings": self.min_crossings,
134         "min_players": self.min_players,
135         "classifier_model": self.classifier_model,
136         "feature_names": self.feature_names,
137         "threshold": self.threshold,
138         "merge_gap": self.merge_gap,

```

```

139         "tune_threshold": self.tune_threshold,
140         "calibrate": self.calibrate,
141     }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
151         the
152         same, so a regression is always traceable to a recipe change, never to drift."""
153         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
154         return sha1(blob.encode()).hexdigest()[:12]
155
156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182         ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183         col = vc.features.get(name)
184         n = len(vc.intervals)
185         if col is None:
186             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                             name, vc.name)
188             return [0.0] * n
189         if len(col) != n:
190             raise ExperimentError(
191                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192         return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
196     List[List[float]]:
197         cols = [_feature_column(vc, name) for name in feature_names]
198         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B

```

```

players)."""
202     n = len(vc.intervals)
203     mask = [True] * n
204     if variant.min_crossings > 0:
205         col = _feature_column(vc, "net_crossings")
206         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
207     if variant.min_players > 0:
208         col = _feature_column(vc, "players_in_court")
209         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
210     return mask
211
212
213     def predict(variant: Variant, vc: VideoCandidates,
214                 model: Optional[LogisticRegression] = None,
215                 threshold: Optional[float] = None,
216                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217         """Variant prediction: gate by persisted features, optionally filter by a learned
218         model, then merge. Pure and deterministic.
219
220         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225         them.
226
227         """
228         mask = _gate_mask(variant, vc)
229         if variant.feature_names:
230             if model is None:
231                 if variant.classifier_model is None:
232                     raise ExperimentError(
233                         f"variant {variant.name!r} is learned but has no model to predict "
234                         f"with (pass a fitted model or set classifier_model)")
235                 model = LogisticRegression.load(variant.classifier_model)
236                 proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
237 variant.feature_names))]
238             if calibrator is not None:
239                 proba = [calibrator(p) for p in proba]
240             thr = variant.threshold if threshold is None else threshold
241             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
242             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
243             return merge_overlaps(kept, gap_tol=variant.merge_gap)
244
245
246     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
247                             train_videos: Sequence[VideoCandidates], iou: float
248                             ) -> "tuple[Optional[Callable[[float], float]],
249 Optional[float]]":
250         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
251         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
252         (use the variant default). Honest: never looks at the held-out video.
253
254         """
255         calibrator: Optional[Callable[[float], float]] = None
256         if variant.calibrate:
257             raw: List[float] = []
258             y: List[int] = []
259             for vc in train_videos:
260                 raw.extend(float(p) for p in
261                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
262 [])))
263                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
264             if variant.calibrate == "platt":
265                 calibrator = fit_platt(raw, y)
266             elif variant.calibrate == "isotonic":
267                 calibrator = fit_isotonic(raw, y)

```

```

262         else:
263             raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
264         threshold: Optional[float] = None
265         if variant.tune_threshold:
266             best_t, best_f1 = variant.threshold, -1.0
267             for k in range(1, 20):                                     # grid 0.05..0.95 on the train
fold's rally F1
268                 t = round(0.05 * k, 2)
269                 f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
270                             vc.gts or [], iou).f1 for vc in train_videos]
271                 mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272                 if mean_f1 > best_f1:
273                     best_f1, best_t = mean_f1, t
274                 threshold = best_t
275             return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
297         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
mean aggregate.
298
299
300         Prediction policy:
301         - **static variant** (no learned filter): predict each video directly ? no
training,
302         so no leakage; per-video scoring is already honest.
303         - **learned, pinned model** (``classifier_model`` set): score every video with the
SHIPPED model. ? optimistic if those videos were in its training set ? use this
304         to
305         report "how the shipped artifact does here", not for the promotion decision.
306         - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
307         train on the OTHER videos, predict the held-out one. The number to trust.
308         - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
309
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}

```

```

317
318     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319     pinned_model = (LogisticRegression.load(variant.classifier_model)
320                     if variant.classifier_model is not None else None)
321     all_model = None
322     if recipe_lovo and not honest:
323         all_model = _train(variant, videos, iou)
324
325     for i, vc in enumerate(videos):
326         cal: Optional[Callable[[float], float]] = None
327         thr: Optional[float] = None
328         if recipe_lovo and honest:
329             others = [v for j, v in enumerate(videos) if j != i]
330             if not others:
331                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332             model: Optional[LogisticRegression] = _train(variant, others, iou)
333             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                 assert model is not None # _train never returns
None
335                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                     for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])

```

```

372         got = _segm.f1_at_overlaps(preds, gts, overlaps)
373         for ov in overlaps:
374             f1k[ov].append(got[ov])
375         ratios.append(_segm.segment_count_ratio(preds, gts))
376
377     def _mean(xs: List[float]) -> float:
378         return sum(xs) / len(xs) if xs else 0.0
379     out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380     out["segment_count_ratio"] = _mean(ratios)
381     return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                             "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}

```

```

424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]]
430     ]
431
432     # One fingerprint over the whole label set ? detects "the deltas were measured
against
433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451     # ----- SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455         agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461
462         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465           the human / a later golden label adjudicates).
466
467         Both variants must be predictable without labels: a learned variant therefore needs
a
468         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
469         ``metrics.match_intervals`` ? no new matching logic.
470         """
471         preds_a = predict(variant_a, video)
472         preds_b = predict(variant_b, video)
473         mr = match_intervals(preds_a, preds_b, agree_iou)
474
475         agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
476             for m in mr.matches]
477         agree_iou = [m.iou for m in mr.matches]
478         a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
479         b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
480
481         def _dur(ivs: List[Interval]) -> float:
482             return sum(e - s for s, e in ivs)
483
484         return {

```



```

485         "video": video.name,
486         "agree_iou": agree_iou,
487         "variant_a": variant_a.name,
488         "variant_b": variant_b.name,
489         "num_a": len(preds_a),
490         "num_b": len(preds_b),
491         "num_agreed": len(agreed),
492         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
493         "duration_a": _dur(preds_a),
494         "duration_b": _dur(preds_b),
495         "agreed": agreed,
496         "a_only": a_only,
497         "b_only": b_only,
498     }
499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:
533             entries = json.load(f)
534     entries.append(row)
535     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
536     with open(path, "w", encoding="utf-8") as f:
537         json.dump(entries, f, indent=2)
538     logger.info("recorded experiment %s vs %s (?f1=%3f) -> %s",
539               row["variant_a"]["name"], row["variant_b"]["name"],
540               (result.get("delta") or {}).get("f1", 0.0), path)
541     return row
542
543 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                          gts: Optional[List[Interval]] = None,
545                          feature_names: Sequence[str] = ()) -> VideoCandidates:
546     """Bridge persisted detection ? `VideoCandidates` via
547     `Database.query_candidate_segments`.
548 
```

```

548     ``feature_names`` selects which persisted features to lift out of each row's
``stats``
549     JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550     function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551     but works against any `Database` (including an in-memory one in tests).
552     """
553     rows = db.query_candidate_segments(video_id, detector=detector)
554     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555     features: Dict[str, List[float]] = {}
556     for name in feature_names:
557         if name == "duration":
558             features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559         else:
560             features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561     return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

14. user (2026-06-14T08:24:30.224Z)

```

1     """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3     The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
a
4     single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
training
5     and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
6     ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
7     turns those into an honest summary:
8
9     - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
10     interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
11     many *videos* it wins on (the distribution-free test the repo already leans on:
12     "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
noise.
13     - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
learned
14     producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
15     match, never a held-out window from the same video).
16
17     See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18     deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19     ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20     """
21     from __future__ import annotations
22
23     import math
24     import random
25     import statistics
26     from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28     Interval = Tuple[float, float]
29     FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31
32     def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33                     n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34         """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
35

```

```

36     ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
to
37     resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
band.
38     """
39     vals = [float(v) for v in values]
40     n = len(vals)
41     if n == 0:
42         return (0.0, 0.0)
43     if n == 1:
44         return (vals[0], vals[0])
45     rng = random.Random(seed)
46     means: List[float] = []
47     for _ in range(max(1, n_boot)):
48         means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
49     means.sort()
50     alpha = (1.0 - confidence) / 2.0
51     last = len(means) - 1
52     lo = means[max(0, int(math.floor(alpha * last)))]
53     hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
54     return (lo, hi)
55
56
57     def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
58                     n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
59         """{mean, std, n, ci_low, ci_high, confidence} ? report this, never a bare mean
(C1)."""
60         vals = [float(v) for v in values]
61         n = len(vals)
62         mean = statistics.mean(vals) if vals else 0.0
63         std = statistics.pstdev(vals) if n > 1 else 0.0
64         lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
65         return {"mean": mean, "std": std, "n": float(n),
66               "ci_low": lo, "ci_high": hi, "confidence": confidence}
67
68
69     def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
70         """Two-sided exact sign test over per-fold deltas (B ? A).
71
72         ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
73         excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
74         distribution-free "did B win on enough *videos*?" check.
75         """
76         wins = sum(1 for d in deltas if d > eps)
77         losses = sum(1 for d in deltas if d < -eps)
78         ties = len(deltas) - wins - losses
79         n_eff = wins + losses
80         if n_eff == 0:
81             p = 1.0
82         else:
83             k = min(wins, losses)
84             tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
85             p = min(1.0, 2.0 * tail)
86         return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
87               "n_effective": float(n_eff), "p_value": p}
88
89
90     def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                        confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                        eps: float = 0.0) -> Dict[str, Any]:
93         """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95         ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
by
96         fold (same video order). A promotion is honest when the delta CI clears 0 *and* the

```

```

sign
97     test agrees ? not when a single mean ticked up.
98     """
99     n = min(len(a_values), len(b_values))
100    deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
101    summary = mean_with_ci(deltas, confidence, n_boot, seed)
102    return {
103        "mean_delta": summary["mean"],
104        "ci_low": summary["ci_low"],
105        "ci_high": summary["ci_high"],
106        "n": float(n),
107        "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
108        "sign_test": paired_sign_test(deltas, eps),
109    }
110
111
112    def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
113        """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
unit.
114
115        The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
116        first-seen (deterministic). Groups with no complement (a single group) yield no
fold.
117        """
118        seen: List[Any] = []
119        for g in groups:
120            if g not in seen:
121                seen.append(g)
122        n = len(groups)
123        folds: List[Tuple[List[int], List[int]]] = []
124        for g in seen:
125            test = [i for i in range(n) if groups[i] == g]
126            train = [i for i in range(n) if groups[i] != g]
127            if test and train:
128                folds.append((train, test))
129        return folds
130
131
132    def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133        """Out-of-fold predictions for honest stacking (C3).
134
135        For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136        other groups and returns predictions for ``test_idx`` (in that order); results are
137        scattered back to a list aligned to the input items. So a *learned* producer's
outputs,
138        when used as arbiter features, never include in-sample predictions ? the leakage
path
139        that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140        (needs >= 2 groups to predict).
141        """
142        n = len(groups)
143        oof: List[Optional[Any]] = [None] * n
144        for train_idx, test_idx in grouped_folds(groups):
145            preds = fit_predict(train_idx, test_idx)
146            for pos, ti in enumerate(test_idx):
147                if pos < len(preds):
148                    oof[ti] = preds[pos]
149        return oof
150

```

Now let me look at the correction loop (the per-user label source) and the per-video calibration. Let me also check the owned-model training study for the on-device/compression analysis, and the n=6 LOVO numbers.

16. user (2026-06-14T08:24:35.314Z)

```
Found 12 files
backend\main.py
backend\infrastructure\database.py
backend\eval\fusion_golden.py
backend\eval\score_calibration.py
backend\eval\segmentation_metrics.py
backend\eval\eval_stats.py
backend\tools\export.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\eval\harness.py
backend\reporting\harvest.py
backend\reporting\spec.yaml
```

17. user (2026-06-14T08:24:35.355Z)

```
docs\archives\COMMERCIAL_READINESS_REVIEW.md
docs\archives\MONETIZATION_AUDIT.md
docs\archives\README.md
docs\archives\checkpoints\2026-06-eval-baseline\README.md
docs\archives\checkpoints\2026-06-logging-reporting\README.md
docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
docs\archives\past_projects\README.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\archives\checkpoints\2026-06-strategy-foundation\README.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\quality-iterations\2026-06-initial\README.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
docs\archives\decisions\DECISIONS.md
docs\archives\TEST_RUN_SUMMARY.md
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
docs\archives\quality-iterations\README.md
```

18. user (2026-06-14T08:24:38.605Z)

```
1      # Quality Iteration: 2026-06 Multi-video LOVO + learned-segmenter prototype
2
3      **Date:** 2026-06-08
4      **STATUS: COMPLETED** (second concrete iteration; follows `../2026-06-human-gt-tuning/`)
5
6      > The session that turned "tune one match" into "is the *method* right across matches."
Built a 2-match
7      > human-labeled corpus, ran the first honest cross-video LOVO, uncovered that the
unsupervised windowing
8      > heuristic is **footage-fragile even when WASB recall is excellent**, and **proved with
a prototype that the
9      > rally signal is learnable** where the heuristic fails. Motivates + de-risks the owned
model
10     > (`../research/OWNED_MODEL_TRAINING_STUDY.md`).
11
12     ## Corpus built this iteration
13     | video | role | rallies | res / fps | trajectory |
14     |---|---|---|---|---|
15     | Adarsh-and-Avi (full) | train | **96** | singles · 1080p / 59.94 |
```

```

~/clips/adarsh_avi_full_traj.csv` (84,480 fr) |
16 | Kushagra-Yash-Avi | train | 32 | singles · 4K?1920 / 59.94 |
~/clips/kushagra_traj.csv` (32,232 fr) |
17 | **LargeTestVideo** | **train** | **49** | **doubles · 5.3K?1920 / 29.97** |
~/clips/largetestvideo_traj.csv` (23,040 fr) |
18 | **gBaaddy (20 May)** | **train** | **47** | **1080p / 59.94 · multi-court** |
~/clips/gbaaddy_traj.csv` (52,232 fr) |
19 | **TestLargeVideo** | **train** | **6** | **5.3K?1920 / 29.97 · short** |
~/clips/testlarge_traj.csv` (3,019 fr) |
20
21 *(Grew to **n=5 distinct matches** over the session: 2 singles + 1 doubles + 2 more,
across 3 cameras/frame-rates,
22 incl. 2 multi-court ? see the n=3 and n=5 updates below.)*
23
24 All WASB trajectories generated on one 8 GB GPU, **serialized via file-based wait-
loops** (`run_wasb_.sh`);
25 GPU never double-booked, race-free via a `kushagra_traj.csv`-exists gate). Manifest:
`output/human_lovo_manifest.json`.
26
27 ## Result 1 ? full-match Adarsh (96 rallies, IoU?0.5)
28 baseline F1 **0.632**, tuned+gate2 **0.727** (P0.80/R0.67), sweep ceiling **0.750**
(vel0.02/merge1.0/min_dur2.0/gate off).
29 vs the opening-10-min slice (40 rallies, prior iteration): 0.667 / 0.742. **Best-config
region is identical across the
30 slice and the full match** (merge_gap 1.0, min_dur 2.0; velocity irrelevant) ? tuning is
stable. Per-rally:
31 `output/adarsh_avi_full_vs_human.csv`.
32
33 ## Result 2 ? first cross-video LOVO (`calibrate_local`, the number to trust)
34 ...
35 per-video best-fit: adarsh 0.777 | kushagra 0.163 (ceiling ? NO config beats it)
36 leave-one-video-out: baseline mean 0.316 ? calibrated 0.443 ± 0.279 (overfit gap
+0.027, tiny)
37 held-out adarsh 0.722 (config tuned on Kushagra ? Adarsh's own optimum ? config
TRANSFERS)
38 held-out kushagra 0.163 (config tuned on Adarsh = Kushagra's own ceiling)
39 ...
40 **The asymmetry is the finding.** Tuning generalizes fine *where the heuristic can solve
the video*
41 (Kushagra?Adarsh = 0.722). But **Kushagra is unsegmentable by *any* velocity-windowing
config (0.163)** ? and
42 this is NOT a WASB-recall failure: WASB tracks the shuttle **better** on Kushagra than
Adarsh (**97% of GT
43 rallies contain ?2 net-crossings**, median 5, 310 visible pts/rally, vs Adarsh 76%). The
likely mechanism:
44 Kushagra's shuttle is visible 67.5% of the time (vs Adarsh 40%) ? tracked *through*
inter-rally activity ? the
45 "shuttle-is-moving" heuristic can't place clean boundaries (mean boundary error ~5 s).
**The ±0.279 spread is
46 method variance, not config.** This also **corrects the earlier "~0.73 baseline"** ?
that was fit-on-self; the
47 honest cross-video heuristic bar is **~0.44.**
48
49 ## Result 3 ? learned per-frame prototype (the proof) ?
`backend/eval/rally_seq_proto.py`
50 A dependency-light prototype (numpy + scipy + the repo's pure-Python
`LogisticRegression`): 6 hand-built per-frame
51 features over the WASB trajectory ? visibility rate, mean/max shuttle speed, **net-
crossing rate**, x/y spread ?
52 labeled rally(1)/non-rally(0), **honest leave-one-video-out** (train on one match, test
the other; split by match).
53
54 | held-out | per-frame AUC | learned F1 (real LOVO) | learned F1 (oracle threshold) |
heuristic ceiling |
55 |---|---|---|---|---|

```

```

56 | Adarsh | 0.878 | 0.704 | 0.721 | 0.73 |
57 | **Kushagra** | **0.841** | 0.049 | **0.578** | **0.163** |
58
59 **The signal is learnable.** Held-out per-frame AUC is **0.84?0.88** ? including on
Kushagra (trained only on
60 Adarsh), the match the heuristic *cannot* segment. With a calibrated threshold the
learned model hits
61 **Kushagra F1 0.578 ? 3.5× the heuristic's 0.163 ceiling.** The naive cross-video
threshold gives 0.049, so the
62 remaining gap is **threshold/operating-point calibration at n=2, not representation**
(the probability scale
63 shifts across footage; fixed by more videos / per-video calibration / a scale-invariant
sequence model).
64
65 > This directly answers the owned-model study's "dominant empirical unknown" ? *can a
low-dim WASB trajectory
66 > feed a learned temporal segmenter?* **Yes (AUC 0.84).** The prototype is the seed of
the study's **Gen-0
67 > (ASFormer-over-WASB-features)** ? It is a *directional* proof, not production: n=2,
linear model, hand
68 > features, threshold-transfer unsolved. The real ship gate is a segment-level A/B on
the golden set at n?5
69 > (needs the GPU/WSL machine ? not runnable in this dev container).
70
71 ## Update ? n=3: doubles match added ? **the crossover** (the final number)
72
73 Added **LargeTestVideo** (doubles, 5.3K?1920, **29.97 fps**, 49 rallies, fully tagged) ?
a different play
74 type *and* camera *and* frame rate. Single-video: heuristic best F1 **0.750** (tuned, R
0.918), best-fit 0.827
75 ? so **doubles is heuristic-friendly; Kushagra remains the lone heuristic failure.**
Per-rally:
76 `output/largetestvideo_vs_human.csv`. Manifest now n=3
(`output/human_lovo_manifest.json`).
77
78 **Honest cross-match LOVO, heuristic vs learned, n=2 ? n=3:**
79
80 | held-out match | heuristic LOVO | learned LOVO (AUC) | learned LOVO F1 |
81 |---|---|---|---|
82 | Adarsh (singles) | 0.725 | 0.878 | 0.701 |
83 | **Kushagra (singles, the hard one)** | **0.151** | 0.850 | **0.417** |
84 | LargeTest (doubles) | 0.755 | **0.914** | 0.729 |
85 | **mean held-out F1** | **0.544 ± 0.278** | ? | **0.615** |
86 | *(same metric at n=2)* | *0.443* | ? | *0.377* |
87
88 **Findings (the data-backed case, honestly):**
89 1. **The learned model OVERTAKES the heuristic at n=3** (mean **0.615 vs 0.544**); at
n=2 it *trailed*
90 (0.377 vs 0.443). The crossover the plan predicted ? the learned path **scales with
data**, the heuristic
91 does not.
92 2. **Both means rose with n, for opposite reasons.** The heuristic rose only because the
new match (doubles)
93 happens to suit it; it **still cannot rescue Kushagra at any n** (held-out 0.151,
capped). The learned model
94 rose because **more data closed its calibration gap** ? Kushagra's held-out F1 went
**0.049 ? 0.417** just
95 from one added match (oracle ceiling 0.593, AUC 0.850 ? it will keep climbing with
n).
96 3. **The decisive edge is on the *hard* footage** ? where the heuristic fails
(Kushagra), the learned model is
97 **2.8× better and rising** ; on the easy matches (Adarsh, doubles) they're comparable
(~0.72?0.75). The owned
98 model's value is exactly the generalization the heuristic lacks.
99 4. **Generalizes across play type** ? the **doubles** match, held out and trained only

```

on the **two singles**

100 matches, scores AUC **0.914** / F1 0.729. Different player count, camera, and frame rate, and the per-frame

101 rally signal still transfers. (Notably *despite* the prototype's speed features not yet being fps-normalized

102 ? 30 vs 60 fps ? which is a known refinement that should only help.)

103

104 **Final quality metrics (n=3, honest leave-one-video-out)**

105 - **Heuristic (current pipeline):** mean held-out F1 **0.544 ± 0.278** ? the ±0.278 is the footage-fragility

106 (Kushagra ~0.15 vs others ~0.75); **hard-capped**, does not improve on the videos it can't represent.

107 - **Learned prototype (owned-model seed):** mean held-out F1 **0.615**, per-frame **AUC 0.857±0.91**, **0.24**

108 for one added match and still climbing. This is a *directional* number (n=3, linear model, hand features,

109 fps-norm + per-video calibration still on the table) ? the trend, not the absolute, is the signal.

110

111 **Conclusion (reassures the plan ? with data, not optimism):** the owned/learned path has **already overtaken**

112 the heuristic at n=3 and improves with every match, while the heuristic is **ceilinged** by footage it cannot

113 represent. The bottleneck is confirmed to be **data scale + calibration (n?5)**, not method. Reusing every

114 labeled video for training is the right strategy ? each one lifts the learned model and none can fix the

115 heuristic's hard cases. ? Proceed with `../../research/OWNED_MODEL_TRAINING_STUDY.md` and

116 `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (Gen-0 first), gated on n?5 + owner sign-off.

117

118 **Update ? n=5 + the MULTI-COURT root cause (the defining finding)**

119

120 Added **gBaaddy** (47 rallies, 1080p/60) + **TestLargeVideo** (6 rallies, 5.3K/30 ? *too short to be a reliable*

121 *fold*). Corpus now n=5. Single-video heuristic: gBaaddy **0.277**, TestLarge 0.316 ? both low. gBaaddy is 1080p/60

122 *like Adarsh* yet fails, so it's **not** a resolution/fps issue.

123

124 **Root cause ? confirmed with data (owner's insight):** background games

125 WASB tracks **every** shuttle in frame, including **other courts**. Videos with background games have high shuttle

126 visibility *during the foreground game's dead time* ? the velocity-windowing heuristic can't place clean boundaries.

127 The **"contrast"** metric (in-rally ÷ dead-time shuttle visibility) cleanly predicts heuristic success:

128

match	heuristic best F1	in-rally vis	dead-time vis	contrast	court
Adarsh (singles)	0.73	69%	18%	3.9×	single
LargeTest (doubles)	0.75	78%	22%	3.6×	single
Mahadevpura (singles, +n=6)	0.67	74%	33%	2.2×	borderline (some background)
gBaaddy	0.28	78%	41%	1.9×	multi (background games)
Kushagra	0.16	82%	57%	1.4×	multi (background games)

136

137 The contrast metric predicts heuristic F1 **monotonically across 5 matches** (3.9×±0.73 ? 1.4×±0.16) ? Mahadevpura's

138 intermediate 2.2× ? 0.67 lands exactly on the curve. 3.6× ? heuristic works; 1.9× ? fails; ~2× ? borderline. **Academies are multi-court by definition**, so Kushagra/gBaaddy are the

139 **most market-representative** videos ? "focus on the prominent game" is a core product requirement, not an edge case.

140 **Lever = foreground-court isolation** (spatial gate to the prominent court before windowing). The dead-time


```

141     detections are laterally offset (side courts), but isolation needs **real court-region
detection** ? a naive centroid
142     is muddled because the foreground shuttle also rests low between points. Near-term, no
new model; also a *design
143     requirement* for the e2e owned model (which would learn foreground attention natively).
144
145     ### Learned prototype LOVO (n=5)
146     | held-out | per-frame AUC | learned F1 (LOVO) | oracle F1 | heuristic best |
147     |---|---|---|---|---|
148     | Adarsh (singles) | 0.879 | 0.709 | 0.722 | 0.73 |
149     | Kushagra (multi) | 0.853 | **0.561** | 0.633 | 0.16 |
150     | LargeTest (doubles) | 0.915 | 0.713 | 0.756 | 0.75 |
151     | gBaaddy (multi) | 0.833 | **0.574** | 0.574 | 0.28 |
152     | TestLarge (6 rallies) | 0.726 | 0.286 | 0.471 | 0.32 |
153     | **mean** | ? | **0.568** | ? | ? |
154
155     - **Recovers BOTH multi-court videos** (Kushagra 0.561, gBaaddy 0.574 vs heuristic
0.16/0.28) ? the learned model's
156     spatial-spread features (`x_std`/`y_std`) help discount background courts.
157     - **Kushagra keeps climbing with n:** 0.049 ? 0.417 ? **0.561**.
158     - ? **Honest mean nuance:** the learned mean *dipped* 0.615 (n=3) ? 0.568 (n=5) ?
**entirely the 6-rally TestLargeVideo
159     fold** (F1 0.286, far too short to read). **Excluding it: 0.639** ? above n=3. The
mean is a blunt tool across folds
160     ranging 6?96 rallies; the **per-video learned-vs-heuristic gap on the hard videos is
the real signal**, and it's decisive.
161
162     ### Final quality metrics & conclusion (n=6)
163     **Honest cross-video LOVO mean, heuristic vs learned, by corpus size:**
164
165     | | n=2 | n=3 | n=5 | **n=6** |
166     |---|---|---|---|---|
167     | heuristic (`calibrate_local`) | 0.443 | 0.544 | 0.463 | **0.480 ± 0.242** |
168     | learned prototype | 0.377 | 0.615 | 0.568 | **0.583** (~0.64 excl. 6-rally fold) |
169
170     - **The learned model has led since n=3 and holds ~+0.10** (n=6: learned 0.583 vs
heuristic 0.480). It stays robust as
171     the corpus diversifies; the heuristic stays capped by the multi-court videos it can't
represent (per-fold n=6: Adarsh
172     0.751, doubles 0.737, Mahadevpura 0.657, but Kushagra **0.157** / gBaaddy **0.309**).
173     - **n=6 added MahadevpuraSingles** (borderline multi-court, contrast 2.2×): held out and
trained on the *other 5*, the
174     learned model hit **AUC 0.902 / F1 0.712** (vs heuristic 0.657) ? clean generalization
to an unseen match.
175     - **The decoupling is the headline:** learned held-out **AUC is 0.83?0.92 on every match
*regardless of contrast**,
176     while the heuristic swings 0.16?0.75 *with* contrast. The learned model has
**decoupled from the multi-court problem**
177     the heuristic can't escape.
178     - **The #1 near-term lever is the multi-court confound** (foreground-court isolation) ?
both the heuristic's biggest
179     failure and the product's core requirement. Bottleneck = **data scale + calibration +
multi-court isolation, not method.**
180
181     ## Heuristic discovery for multi-court (what we tried, what's promising)
182
183     Motivated by the contrast finding, two label-free trajectory pre-filters were prototyped
+ measured at the tuned
184     operating point across all 5 videos (scripts not committed ? discovery):
185
186     | candidate | multi-court ? (Kushagra/gBaaddy) | clean ? (Adarsh/doubles) | verdict |
187     |---|---|---|---|
188     | **Foreground spatial gate** (keep dominant 2D-density court region) | +0.03 / +0.03 |
**?0.13 / ?0.12** | ? net ?0.04 ? helps multi-court a little but removes legit wide foreground
points on clean videos |

```

189 | ****Track-continuity filter**** (drop implausible position jumps) | 0.00 | 0.00 | ? no-op
? the confusion isn't transient jumps; WASB tracks a court coherently, switching at gaps |

190

191 ****Conclusion:**** simple single-cue trajectory heuristics ****cannot**** isolate the
foreground court ? courts aren't

192 separable by raw density, and the learned model already handles multi-court better (AUC
0.83) by combining cues.

193

194 ****Audio for foreground discrimination ? TESTED (2026-06-08), WEAK ?.**** Hypothesis: the
GoPro mic sits at the

195 foreground court, so foreground hits are loud/near and background games quiet/distant ?
audio energy gates ****is the**

196 prominent game active?" ? the signal the visual trajectory can't give. ****Probed on the**
2 multi-court videos + a clean

197 control****** (extract audio ? onset-flux + RMS energy, AUC of in-rally vs dead-time):
onset/RMS AUC only ****0.53?0.65****;

198 audio contrast 1.1?1.9 $\times$ (below the visual contrast). High-frequency bands (8?22 kHz @ 48
kHz) raise the ***contrast***

199 (Kushagra 1.1?1.7 $\times$, gBaaddy 1.3?1.9 $\times$? confirming HF-attenuates-with-distance physics)
but ****AUC stays ~0.6****. ****Why it**

200 fails:****** "dead time" isn't quiet ? the FOREGROUND court is loud during dead-time too
(footwork, talk, shuttle pickup) +

201 gym reverb + background games, so audio loudness ? ***court occupancy***, not ***rally***. Same
wall as E1 (~2.2 $\times$ ceiling).

202 ****Verdict:** audio alone is NOT the multi-court gate.****** (A learned timbre/rhythm model or
audio ***fused into*** the learned

203 segmenter might add marginal value, but the visual learned model already hits AUC 0.85
on these videos.)

204

205 ****Court-region detection ? SMOKE-TESTED (2026-06-08), LOW HEADROOM ?.**** Feasibility
probe: of the DEAD-TIME shuttle

206 detections, what fraction fall OUTSIDE the foreground rally region (= separable
background courts a court gate could

207 remove)? ****Only 12?14% on the multi-court videos**** (Kushagra 12%, gBaaddy 14%) ?
****86?88% of the dead-time confusion is**

208 INSIDE the foreground court region.****** A ***perfect*** court gate lifts contrast only to
~1.6?2.2 $\times$ (Kushagra 1.4?1.6, gBaaddy

209 1.9?2.2) ? still below the ?3.6 $\times$ "works" threshold. So the confusion is ****not****
spatially-separable background courts;

210 it's the ****foreground court's own between-point activity**** (footwork, serves, shuttle
pickup) and/or background courts

211 that overlap the foreground image area. ? ****Court-region detection (heuristic OR a**
dedicated model) would NOT

212 meaningfully help.****** (Doesn't need an own model ? but isn't worth building either.)

213

214 **###** The verdict on heuristics: it's a TEMPORAL problem, only the learned model solves it

215 Three heuristic ideas were tested this session and all came up short ? ****foreground**
spatial gate****** (net ?0.04),

216 ****audio energy**** (AUC ~0.6), ****court-region separability**** (12?14% out-of-region, low
headroom). They fail for the SAME

217 reason: the multi-court dead-time confusion is ****rally-vs-non-rally activity in the**
same place and frequency band******, not

218 a spatial or loudness difference. Distinguishing it needs the learned model's
****temporal/structural**** features (sustained

219 net-crossing rhythm, activity duration, spatial-spread dynamics) ? which is exactly why
the prototype already hits ****AUC**

220 0.83****** on Kushagra/gBaaddy where every hand-crafted heuristic fails. ****There is no cheap**
heuristic shortcut; the learned

221 segmenter is the path.****** (A multi-feature heuristic combining cross-rate + speed +
spread would just be re-deriving the

222 prototype by hand ? so prefer the prototype.)

223

224 **##** Learnings (carry forward)

225 - ****The bottleneck is the method + data, not WASB recall or labels.**** WASB delivers
strong shuttle signal on both

```

226     matches; the unsupervised heuristic extracts it on one and not the other. A learned
model does (AUC 0.84).
227     - **Tuning is real but footage-bounded** ? `merge_gap 1.0 / min_dur 2.0` transfers; the
gate trades recall?precision;
228     velocity is ~irrelevant. But no config rescues a video the heuristic class can't
represent.
229     - **Calibration, not representation, is the n=2 wall** ? learned probabilities separate
(AUC 0.84) but the operating
230     point doesn't transfer across footage with one training video. **n?5 distinct matches
is the highest-ROI unblock.**
231     - **Group by match, not file** for any cross-val (Adarsh-main + Adarsh-last-game = one
unit; `calibrate_local` has no
232     grouping, so keep one entry per distinct match).
233     - **Reuse the labels** ? Kushagra's labels + trajectory are valuable *training data for
the learned model* even though
234     the current pipeline scores 0.16 on it. Validated empirically.
235
236     ## Reproduce
237     ```bash
238     # trajectories: bash ~/clips/run_wasb_{avi_full,kushagra,gbaaddy}.sh    (serialized on
one GPU)
239     python -m backend.eval.eval_partial_labels --trajectory
"%TEMP%/adarsh_avi_full_traj.csv" \
240     --labels "?/Adarsh and Avi.rallies.csv" --fps 59.94 --frame-width 1920      #
full-match + --sweep
241     python -m backend.eval.calibrate_local --manifest output/human_lovo_manifest.json
--metric f1 # heuristic LOVO
242     python -m backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
# learned prototype
243     ```
244
245     ## What's next
246     - **? Foreground-court isolation experiment** (the multi-court fix) ? spatially gate the
trajectory to the prominent
247     court before windowing; needs real court-region detection (densest player/shuttle
cluster, court-line detection, or
248     largest-court region), NOT a naive centroid. Would likely fix the heuristic on
Kushagra/gBaaddy AND clean the learned
249     features. Near-term, no new model. Highest-value lever given the academy/multi-court
target env.
250     - **Keep growing the corpus** (more varied matches tomorrow) ? re-run LOVO + prototype;
per-video threshold calibration
251     as n grows. Add fps-normalization to the prototype's speed features.
252     - Build the corpus format + the ASFormer Gen-0 A/B (owner sign-off; per
`../../research/OWNED_MODEL_TRAINING_STUDY.md`
253     + `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`, after scaffolding).
254
255     ## Related
256     - Prior iteration: `../2026-06-human-gt-tuning/` . Study:
`../../research/OWNED_MODEL_TRAINING_STUDY.md`
257     - Code: `backend/eval/{eval_partial_labels,calibrate_local,rally_seq_proto}.py` .
Checkpoints: `../../checkpoints/2026-06-eval-baseline/`
258

```

19. user (2026-06-14T08:24:38.819Z)

```

1     # First end-to-end A/B experiment on the golden corpus (n=6)
2
3     **STATUS: COMPLETED (2026-06-14).** The first time the experiment harness
4     ([EXPERIMENT_HARNESS.md](../../EXPERIMENT_HARNESS.md)) + the measurement-honesty
helpers
5     (course corrections C1?C4,
[ANALYZERS_AND_RECONCILERS.md](../../ANALYZERS_AND_RECONCILERS.md) $13)
6     were run end-to-end on real golden data ? a posterity record of the framework's first
real use.

```

```

7
8     **Trigger.** After landing the signal-fusion framework doc + §13 literature review and
the additive
9     C1?C4 helpers (`backend/eval/eval_stats.py`, `score_calibration.py`,
`segmentation_metrics.py`, PR #124),
10    the question was: *does the framework actually hold up on a golden video? This is the
validation run.
11
12    ## Method (100% offline ? the "persist detection once, re-score forever" path)
13    - **No GPU.** Re-scored the **persisted WASB shuttle trajectories** of the 6 golden
matches
14    (`output/human_lovo_manifest.json`); detection was never re-run.
15    - **One consistent candidate set per video** via `distill_local.build_candidates`
(recall-oriented
16    proposal windowing `velocity_thresh=0.005, merge_gap=1.0, min_dur=0.5`) ? per-window
shuttle features
17    `[duration, peak_velocity, mean_velocity, active_ratio, net_crossings]`; golden labels
via
18    `calibrate_wasb.load_golden_gts`; per-candidate y via the evaluator's own IoU matcher.
19    - **Three variants re-scored on that same set** (`backend/eval/experiment.py`):
20    - **CONTROL** ? merge all candidates (no filter).
21    - **Gate-A** ? keep candidates with `net_crossings` ? 2, then merge (the held-shuttle
/ service-feed
22    structural FP killer; `rally_gate`).
23    - **learned** ? a window-level logistic regression over the 5 shuttle features,
**leave-one-VIDEO-out**
24    (train on the other 5 matches, predict held-out), prob ? 0.5, merge.
25    - **Graded with:** temporal-IoU F1 (per video), **C1** honest aggregation
(`mean_with_ci`,
26    `paired_delta_ci`, exact `paired_sign_test`), **C4** segmentation metrics
(`f1_at_overlaps`,
27    `segment_count_ratio`), and a **C3** out-of-fold learned mAP@tIoU.
28    - IoU 0.5. Repro: `scratch/ab_n6_lovo.py` (machine-side; reads the gitignored manifest +
the external
29    golden trajectories/labels); raw result `output/ab_experiment_n6.json`.
30
31    ## Corpus
32    | video | candidates | golden rallies | proposal recall ceiling |
33    |---|---|---|---|
34    | adarsh_avi_singles | 172 | 96 | 0.833 |
35    | kushagra_singles (multi-court) | 20 | 32 | **0.125** |
36    | largetest_doubles | 86 | 49 | 0.816 |
37    | gbaaddy (multi-court) | 82 | 48 | **0.312** |
38    | testlarge_short | 14 | 6 | 0.500 |
39    | mahadevpura_singles | 57 | 31 | 0.645 |
40
41    The multi-court matches (kushagra, gbaaddy) have **low proposal-recall ceilings** ? the
decision stage
42    can never recover rallies the windowing missed (the proposal-recall ceiling pitfall,
§13). Their absolute
43    F1 is windowing-capped, not a decision failure.
44
45    ## Results
46
47    **Per-video held-out F1 (IoU 0.5):**
48    | video | CONTROL | Gate-A | learned (LOVO) |
49    |---|---|---|---|
50    | adarsh_avi_singles | 0.597 | **0.722** | 0.613 |
51    | kushagra_singles | 0.154 | **0.163** | 0.163 |
52    | largetest_doubles | 0.593 | **0.667** | 0.535 |
53    | gbaaddy | 0.231 | **0.274** | 0.222 |
54    | testlarge_short | 0.300 | **0.375** | **0.000** |
55    | mahadevpura_singles | 0.455 | **0.571** | 0.308 |
56    | **mean** | **0.388** | **0.462** | **0.307** |
57

```

58 **A/B 1 ? CONTROL vs Gate-A (static).** ?F1 ** $+0.074$ **, 95% CI ** $[+0.042, +0.104]$ **
(excludes 0),
59 sign test **6 wins / 0 losses, $p = 0.031$ **. ? **A real, statistically-supported win** ?
Gate-A improves F1
60 on *every* golden match.
61
62 **A/B 2 ? CONTROL vs window-level learned logistic (LOVO).** ?F1 ** $?0.081$ **, CI
** $[?0.173, ?0.002]$ **,
63 sign test **2 wins / 4 losses, $p = 0.688$ **. ? **Does not clear the bar** ? this variant
is *worse* than
64 CONTROL on this setup (and the mean is dragged by two big per-video losses; the sign
test is inconclusive ?
65 the two honest views disagreeing is itself a reason to report both).
66
67 **C4 ? over-segmentation-aware metrics (mean across videos):**
68 | variant | F1@10 | F1@25 | F1@50 | segment-count ratio |
69 |---|---|---|---|---|
70 | CONTROL | 0.645 | 0.603 | 0.388 | 1.68 |
71 | **Gate-A** | ** 0.748 ** | ** 0.698 ** | ** 0.462 ** | ** 1.01 ** |
72 | learned | 0.564 | 0.509 | 0.307 | 0.89 |
73
74 Gate-A drives the **segment-count ratio $1.68 ? 1.01$ ** ? it nearly eliminates the over-
segmentation that
75 plain temporal-IoU barely reflected. The learned variant's 0.89 is *under*-segmentation
(it drops real
76 rallies). **C3 out-of-fold learned $mAP@tIoU[.3?.7] = 0.245$ ** (weak ranking ?
corroborates the loss).
77
78 ## Wins / losses / lessons
79 - ? **Gate-A (net-crossings ? 2) is a genuine win** ? $+0.074$ mean F1, 6/6 matches, $p =$
 0.031 , CI clears 0,
80 over-segmentation $1.68 ? 1.01$. Promote-worthy by the honest gate. (Confirms the held-
shuttle owner-bug
81 killer on the whole corpus, offline, no GPU.)
82 - ? **The window-level 5-feature logistic (threshold 0.5) is not a win here** ? $?0.081$,
mixed (2W/4L).
83 `testlarge\_short ? 0.000` (the model predicted *zero* rallies) is a
threshold/calibration artifact, a
84 textbook motivation for **C2 (calibrate cue confidence)** and tuning the threshold
inside the LOVO loop.
85 - ?? **Do not conflate this with the documented learned winner.** The
86 [`2026-06-multivideo-lovo`](../2026-06-multivideo-lovo/README.md) result (learned
0.583 vs heuristic
87 0.480) is a **per-frame prototype** (`rally\_seq\_proto.py`), a *different, stronger*
model than this
88 window-level logistic. This experiment compared a *weaker* learned path against a
strong CONTROL
89 (merge-all of a recall-oriented proposal) ? a different matchup. The takeaway is
methodological, not
90 "learned fusion fails."
91 - ? **The methodology itself worked.** C1 gave a confident **YES** on Gate-A *and* a
clear **NO** on the
92 logistic ? preventing a wrong promotion in **both** directions. A bare-mean report
would have hidden
93 Gate-A's significance and might have over-read the logistic's mean. C4 surfaced the
over-segmentation
94 story tIoU buried. This is exactly the framework holding up.
95 - ? **Multi-court is windowing-capped** ? kushagra/gbaaddy absolute F1 is bounded by the
 $0.125/0.312$
96 proposal-recall ceiling; the decision layer can't fix what the proposal missed
(measure AR@AN separately).
97
98 ## Significance
99 First real A/B on the golden corpus ? the harness + C1?C4 produced an **honest,
promotable finding

```

100      (Gate-A)** and an **honest rejection (this logistic)** , entirely offline. This is the
empirical backbone the
101      paper aim ([`paper-coauthor-aim`]) needs ? honest measured results only. Gate-A becomes
a candidate for the
102      no-regression promotion gate; the logistic's failure mode is the concrete task list for
C2 + threshold-in-loop.
103
104      **Links:** [EXPERIMENT_HARNESS.md](../../../../EXPERIMENT_HARNESS.md) ·
105      [ANALYZERS_AND_RECONCILERS.md](../../../../ANALYZERS_AND_RECONCILERS.md) §13 (C1?C4) ·
106      [QUALITY_ITERATIONS.md](../../../../QUALITY_ITERATIONS.md) · predecessor
107      [`2026-06-multivideo-lovo`](../2026-06-multivideo-lovo/README.md).
108

```

20. assistant (2026-06-14T08:24:41.888Z)

This is rich. Now let me check the owned-model training study for on-device/compression specifics and the correction loop / per-video calibration in the DB. Then I'll web-ground the on-device/continual-learning claims.

21. user (2026-06-14T08:24:43.310Z)

```

1      # Owned-model training study (rally segmentation ? end-to-end multimodal)
2
3      **Date:** 2026-06-08 · **STATUS:** DESIGN BLUEPRINT (no implementation until PLATFORM
P0?P4 + healthy corpus; owner sign-off required).**
4
5      > How to train our **OWNED** rally/highlight model **reusing the videos we label**,
aligned to the
6      > Apache-2.0 / no-paid-LLM-training / privacy-local guardrails. Produced by an 11-agent
research
7      > workflow (ground ? 5 web-research dimensions ? adversarial verification of
costs/licenses/privacy ?
8      > synthesis + completeness critic; 35 sources). Empirical results from THIS session
(LOVO + a learned
9      > prototype) are folded in and **resolve the study's biggest open unknown** ? see §"What
we proved today".
10     > Numbers from web sources are 2026 snapshots in a ±15?25% band; **re-quote live before
budgeting.**
11
12     ## Update (2026-06-09, post-PR-#61-review) ? authoritative roadmap is now
`docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`
13     A verified strategy workflow + owner review refined this study. Key changes (the plan
supersedes where they differ):
14     - **VISION (foregrounded):** the goal is **rally/shot/player analytics + highlights for
ALL net-separated racquet
15     sports** (badminton/tennis/TT/pickleball/padel) ? eventually all sports, as **ONE
sport-agnostic, conversational
16     model**. This does **not** change the model/framework picks; it scales **data + per-
sport trajectory detectors** (the
17     accepted "good problem") and adds a licensing front (clean per-sport detectors).
Single-model temporal generalization
18     is an **open hypothesis** (corpus is badminton-only; the once-cited RacketVision
evidence was **refuted** ? it's
19     ball-tracking, not temporal).
20     - **Framework = ms-swift (Apache-2.0)** (only forge with native video+audio+timestamp-
grounding+video-GRPO).
21     **Base = Qwen3-VL (Apache-2.0, uniform sizes, timestamp emission)** ? primary,
**replacing the Qwen2.5-VL-7B framing**.
22     - ? **Gemma 4 stays AMBER and bench-only** (NOT "confirmed clean" as the table below
says): Apache grant likely but
23     Prohibited-Use posture unconfirmed (counsel) + video/timestamp capability disputed ?
not the primary extractor.
24     - **Start with the TAL head, not the VLM** (VLMs miss strict boundaries ? boundary
accuracy is the product). Head now ? VLM later.
25     - **Immediate goal = a high-P/R extraction *harness*, not on-device** (on-device

```

```

deferred). **Reframe supersedes the on-device-now framing.**
26
27     ## Thesis
28
29     Train the owned model in **three staged generations off ONE model-agnostic labeled
corpus** ? a label
30     collected once feeds every generation, so labels are never re-collected. **The moat is
the consented
31     dataset + the eval harness, never the weights.**
32
33     | Gen | When | Approach | Base / model | Compute |
34     |---|---|---|---|---|
35     | **Gen-0** | now ? unblocks at n?5 matches | tiny **temporal action segmentation
(TAS)** head over a hand-built per-frame feature stack from the WASB trajectory we already
produce; per-frame rally(1)/non-rally(0) ? merge into [start,end] | **ASFormer (MIT, ~1.1M
params)** primary; **ActionFormer (MIT)** A/B challenger (overtakes once corpus = hundreds of
matches) | **local RTX 2070 Super**, trains in minutes, $0 |
36     | **Gen-1 / Option A** | 6?12 mo | concatenate more local cue channels onto the SAME
head: trajectory + audio-onset + pose/court serve-gating; fusion in our rally layer, cues
swappable | same ASFormer/ActionFormer, wider input | local + occasional cloud burst (pseudo-
labeling) |
37     | **Gen-2 / Option B** | 12?24+ mo, after scaffolding + healthy corpus | end-to-end
multimodal VLM: (frames+audio+pose+trajectory) ? rally [start,end] (+semantics); SFT cold-start
then **GRPO/RFT with a temporal-IoU-vs-golden reward** | **Qwen3-VL-4B-Instruct** (Apache-2.0,
on-device) primary; **Qwen2.5-VL-7B** fallback; **Gemma 4 E4B/12B-Unified** (Apache-2.0 + native
video+audio) end-state | rented 24?48 GB A100/L40S (does NOT fit 8 GB box) |
38
39     ## What we proved today (de-risks the blueprint)
40
41     The study's ***"dominant empirical unknown"*** was: *can a low-dim WASB trajectory (?3?4
raw dims) feed a
42     TAS head, given ASFormer's small-data result used 2048-d I3D features?* This session
answered it.
43
44     - **Cross-video LOVO of the current heuristic** (`calibrate_local`, Adarsh-96 +
Kushagra-32, human labels):
45     mean held-out F1 **0.443 ± 0.279**; Adarsh 0.722, **Kushagra 0.163** (its own ceiling
? *no* velocity-windowing
46     config does better). **The ±0.279 spread is method variance, not config.** ? This also
**corrects the study's
47     "~0.73 bar"**: that was a fit-on-self single-match number; the honest cross-video
heuristic bar is **~0.44**,
48     which *lowers* the Gen-0 ship gate.
49     - **A learned per-frame prototype** (`backend/eval/rally_seq_proto.py`: 6 hand-built
features ? visibility rate,
50     mean/max shuttle speed, **net-crossing rate**, x/y spread ? into the repo's
dependency-free LogisticRegression,
51     honest LOVO): **per-frame AUC 0.878 (Adarsh) / 0.841 (Kushagra), held-out.** On
Kushagra ? the match the
52     heuristic *cannot* segment (0.163) ? a calibrated threshold reaches **segment F1 0.578
(3.5×)**. The naive
53     cross-video threshold gives only 0.049, so **the remaining gap is threshold/operating-
point calibration at n=2,
54     not representation.**
55
56     - **n=3 update (doubles match added ? `LargeTestVideo`, 49 rallies):** the learned
prototype **overtakes the
57     heuristic.** Mean held-out LOVO F1 ? learned **0.377 ? 0.615**, heuristic **0.443 ?
0.544**. Kushagra's learned
58     held-out F1 jumped **0.049 ? 0.417** (one added match closed most of the calibration
gap); the **doubles** match,
59     held out and trained only on the two singles, scored AUC **0.914** / F1 0.729
(generalizes across play type +
60     frame rate). The heuristic still cannot rescue Kushagra at any n (0.151, capped).
**The honest "bar to beat" is

```

```

61         thus ~0.54 (n=3), not 0.73 ? and the learned path already clears it on the mean and
keeps rising with n.**
62
63     **Conclusion:** the rally signal IS learnable from the WASB trajectory features, cross-
footage (AUC ~0.85?0.91) ?
64     the heuristic just can't exploit it. This **validates the Gen-0 ASFormer-over-WASB-
features path**, shows the
65     hand-built feature stack the study calls for is the right lever, and confirms the
bottleneck is **data + per-video
66     calibration (n?5 matches), not method.** The prototype is the seed of Gen-0, and the
n=2?n=3 trend (learned mean
67     +0.24, overtaking the heuristic) is the empirical green light.
68
69     ## Platform ranking (privacy + cost lens; the fine-tune is a sub-24h, $10?50 job ?
privacy/ergonomics dominate)
70
71     1. **Modal ? PRIMARY.** Strongest *written* no-access contract (verbatim: "Modal will
never access? the inputs/outputs to your Functions, or any data you store"), ?7-day TTL,
SOC2-II, HIPAA-BAA; per-second scale-to-zero matches retrain-on-each-batch. A100 80 GB
~$2.50/hr, H100 ~$3.95/hr (corrected down from cited). Privacy is the load-bearing reason, not
price.
72     2. **RunPod (Secure Cloud tier ONLY) ? FALLBACK / Option-B scale-up.** Cheapest vetted-
DC self-serve; A100 80 GB ~$1.39?1.49/hr. Community/P2P tier is UNVETTED ? **never** touches
consented video. Confirm AES-256/DPA at booking.
73     3. **Vertex AI / GCP ? ON BENCH.** Only strong-privacy option with an India region, but
it's **capacity-gated** (asia-south1 high-end GPU is invitation-only, H100-class A3 not A100)
and **tuning Gemini yields NO portable on-device weights** ? only tuning an Apache-2.0 base in
Model Garden does. Use only if India residency-at-rest is mandated.
74     4. **Self-hosted RTX 2070 Super 8 GB** ? Gen-0 training + ALL inference + CI only.
**Cannot** QLoRA a 7B video VLM (~20?24 GB). Best privacy (nothing leaves the box).
75     5. **Fireworks** (managed, contractual no-train + ZDR) usable; **Together BLOCKED**
until a signed DPA + explicit no-train clause. AWS SageMaker / Azure ML = heavy for a solo
founder; India-residency only.
76
77     ## Label reuse ? one corpus, three transcoders (the actual moat)
78
79     Promote the rally CSVs from eval-only files to **one model-agnostic corpus**: JSONL, one
row per
80     `(video_id, rally_ordinal)` = `{start, end, ending_reason, sport, source
(human|pseudo|vendor),
81     annotator/provenance, consent/opt-in, label_type, split (BY MATCH not clip),
trajectory_ref}`. The
82     provenance/consent fields are the **enforcement point** for the legal guardrails. Three
deterministic
83     transcoders read it, zero re-labeling:
84     1. **Heuristic/LogReg view** ? already exists
(`backend/eval/training.py::label_candidates` via `metrics.match_intervals`).
85     2. **TAL view** ? ActivityNet JSON (`segment:[start,end]`, `label:'rally'`); our seconds
map 1:1.
86     3. **VLM clip?QA view** ? "list every rally [start,end] in this clip"; `ending_reason` ?
a second QA task.
87
88     **One scorer spine:** reuse `metrics.match_intervals` for heuristic-labeling, TAL eval,
AND the Gen-2 RFT
89     reward ? no metric drift. **Leakage guard:** split **by match, never by clip**.
**Amplification (all
90     guardrail-clean):** open-teacher pseudo-labels (WASB/own-model, confidence-gated ? never
paid-LLM),
91     boundary-preserving temporal augmentation, active learning to spend the Roora budget on
highest-uncertainty
92     matches. ? **The new partial-label windowing**
(`eval_partial_labels.restrict_to_window`) must flow into the
93     TAL/VLM views so prefix-labeled videos aren't mis-scored (a corpus that's partially
tagged is the norm).
94

```



```

95     ## Licensing & privacy guardrails (all respected; verify per-checkpoint)
96
97     - **Bases: Apache-2.0/MIT only.** APPROVED: Qwen2.5-VL-7B/32B (Apache-2.0),
Qwen3-VL-4B/8B (Apache-2.0),
98         Gemma 4 E2B/E4B/12B-Unified (Apache-2.0 + native video/audio), InternVL3 ?38B
(MIT/Apache), ASFormer/
99         ActionFormer/TriDet (MIT). **REJECTED:** Gemma 3 (viral Gemma Terms + PUP),
Qwen2.5-VL-3B (NC research),
100         Qwen-72B/InternVL3-78B (100M-MAU cap), Llama Vision (700M-MAU), DAMO VideoLLaMA3
weights (NC + paid-LLM-trained),
101         TimeChat/VTimeLLM/Momentor (NC/no-license = method-reference only), VideoMAE weights
(CC-BY-NC), `yabufarha/ms-tcn`
102         (Commons Clause ? use `sj-li/MS-TCN2` MIT). ? **Gemma 4 size names =
E2B/E4B/26B-A4B/31B/12B-Unified ? there is no plain "4B".**
103     - **No paid-LLM training data** ? Gemini/OpenAI/Anthropic = inference/eval/teacher-
reference only, **never** a
104         training signal. Gemini silver labels stay out of every transcoder.
105     - **Exclude minors by default + separate training opt-in;** per-user adapters trained
only on that user's
106         consented video (deletion = drop the adapter). Enforced at the corpus-membership layer
before transcoding.
107     - **Privacy lever = local inference** (Gen-0 ~1.1M params; Gen-2 4B quantized). Any
cloud *training* needs a
108         signed DPA + no-train assurance; route only owned/consented footage, never user
uploads.
109
110     ## Cost (spend on labels, not GPUs)
111
112     - **Gen-0:** ~$0 (local 2070 Super; trains in minutes; ~4.5 GB VRAM).
113     - **Gen-2 LoRA:** ~$5?40/run on a rented 24?48 GB GPU (RunPod cheapest; **VRAM/token
cost on real 1080p/60fps
114         clips is UNVERIFIED ? 24 GB is a floor, re-benchmark**). Managed (Fireworks/Together)
~$2?30 *if* text-priced
115         (video tokens can balloon). RFT = compute-heavy (multiple rollouts/sample).
116     - **Inference crossover (the unanchored number):** self-hosted 7B beats Gemini
(~$0.05/10-min clip) **only**
117         above sustained QA volume; below it, **paid Gemini is cheaper** ? paid-primary now,
owned-primary later.
118         **Benchmark $/call before committing** (COMMERCIALIZATION C7).
119
120     ## Open risks & critic caveats (read before acting)
121
122     - **?? WASB checkpoint provenance (C3) propagates.** The study calls WASB "MIT," but
that's the *code*; the
123         distributed `wasb_badminton_best.pth.tar` is **academic-data-trained, provenance
unresolved** (COMMERCIALIZATION
124         C3, still open). Every Gen-0/1 plan uses WASB features as input AND as a pseudo-label
teacher ? a "clean" owned
125         model trained on provenance-unclear features **does not launder the provenance.**
ROADMAP Phase-4 "train own WASB
126         weights" is a **prerequisite, not an aside.**
127     - **Gemma 4 "clean + native-video" rests on post-cutoff secondary sources** ? keep AMBER
until a raw HF LICENSE
128         revision is verified, given the Gemma-3 rejection. Its timestamp-emission recipe is
less documented than Qwen's.
129     - **Existing distilled model was trained on Gemini silver labels** ? the (correct) eval-
only policy now **forbids
130         this; that pipeline must be purged of Gemini-derived training labels** to comply.
131     - **Policy ratifications needed:** guardrail literally says "Apache-2.0 only" but the
approved list includes MIT ?
132         ratify "Apache-2.0/MIT". The MAU-cap rejections (72B/78B) may be over-conservative at
our scale.
133     - **On-device privacy claim is unbenchmarked** ? no tokens/sec for a quantized 4B VLM on
8 GB Turing at
134         1080p/60fps. **Measure before relying on it.**

```

135 - ****Amplification is load-bearing for Gen-2**** ? 128 raw rallies can't reach the ~5k?20k clip?QA band alone; if
136 augmentation + QA-multiplexing + pseudo-labels underdeliver, Option B is under-fed.
Amateur 60fps may need more
137 labels than broadcast-trained literature implies.
138 - ****Dropped option worth keeping:**** InternVideo2 encoder (MIT) + light grounding head ?
the ***only*** data-realistic
139 owned-video experiment available at n?128 today. And: a ****one-time ~\$1?2k 24 GB local card**** (e.g. used 3090)
140 would preserve the privacy lever for ***training*** and dissolve most of the cloud-DPA analysis ? uncoded here.
141
142 **## Open questions (need the GPU/WSL machine-side suite ? not runnable in this dev container)**
143
144 1. Does ASFormer/ActionFormer over the WASB feature stack beat the ****honest ~0.44 LOV0**** bar at n?5 matches?
145 (Today's prototype says the per-frame AUC is there ? 0.84 ? so this is now ***likely***; confirm at segment level.)
146 2. Real VRAM + token cost of fine-tuning Qwen3-VL-4B / Qwen2.5-VL-7B on our actual clip?QA frame counts?
147 3. Working Gemma-4 timestamp-emission recipe + exact checkpoint (E4B vs 12B-Unified)?
148 4. Measured self-hosted-7B \$/call + latency vs Gemini on the target workload (the owned-model ROI crossover)?
149 5. Will Together/Lambda/Azure sign a DPA with explicit no-train for consented video, India-configurable?
150 6. Realistic clip?QA yield from 128 rallies via augmentation + QA-multiplexing + pseudo-labels?
151
152 **## Sequencing (owner directive, non-negotiable)**
153
154 This is a ****design blueprint only.**** It unblocks PLATFORM ****P5**** but starts ****no implementation**** until P0?P4
155 scaffolding (async jobs, storage/compute backends) is rock-solid AND the golden corpus is healthy. Paid Gemini
156 stays the always-ready primary/fallback. The only items safe to act on now ? ****and even these need owner**
157 sign-off before code** ? are (a) the model-agnostic corpus format and (b) growing n?2 ? n?5 matches (a small
158 Roora batch), which unblocks honest tuning immediately and is the ****highest-ROI spend (labels, not GPUs).****
159
160 ***Full agent transcript + 35 sources: workflow `wf\_49aab888-e10` (this session).***
161

22. user (2026-06-14T08:24:45.037Z)

```

109:             confirmed_segment_id INTEGER,
110:             human_label TEXT,
114:             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
279:             "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
299:             query += " AND human_label IS NULL"

```

23. assistant (2026-06-14T08:24:48.799Z)

I have a thorough picture of the repo's substrate. Now let me web-ground the key technical-feasibility claims: on-device/continual personalization, small-data fine-tuning, catastrophic forgetting, and on-device VLM fine-tuning feasibility. Let me load the web tools and run searches.

24. user (2026-06-14T08:24:59.549Z)

Web search results for query: "on-device continual learning catastrophic forgetting small data

`fine-tuning 2025 survey"`

Links: [{"title": "Federated Continual Learning: A Survey on Mitigating Spatial-Temporal Catastrophic Forgetting | Proceedings of the 2025 International Symposium on Machine Learning and Social Computing", "url": "https://dl.acm.org/doi/10.1145/3778450.3778530"}, {"title": "Efficient Deep Learning Infrastructures for Embedded Computing Systems: A Comprehensive Survey and Future Envision", "url": "https://arxiv.org/pdf/2411.01431"}, {"title": "GitHub - Wang-ML-Lab/llm-continual-learning-survey: [CSUR 2025] Continual Learning of Large Language Models: A Comprehensive Survey · GitHub", "url": "https://github.com/Wang-ML-Lab/llm-continual-learning-survey"}, {"title": "[2504.13822] Parameter-Efficient Continual Fine-Tuning: A Survey", "url": "https://arxiv.org/abs/2504.13822"}, {"title": "Continual Learning and Catastrophic Forgetting", "url": "https://arxiv.org/pdf/2403.05175"}, {"title": "mSFT: Addressing Dataset Mixtures Overfitting Heterogeneously in Multi-task SFT", "url": "https://arxiv.org/pdf/2603.21606"}, {"title": "A Comprehensive Survey on Continual Learning in Generative Models", "url": "https://arxiv.org/html/2506.13045v3"}, {"title": "[2504.01241] Catastrophic Forgetting in LLMs: A Comparative Analysis Across Language Tasks", "url": "https://arxiv.org/abs/2504.01241"}]

Based on the search results, here are the key findings related to on-device continual learning, catastrophic forgetting, small data fine-tuning, and recent survey work:

On-Device Continual Learning and Catastrophic Forgetting

On-device continual learning performs local training and does not need to send back newly collected data to remote servers, which protects data privacy and ensures data security. However, on-device continual learning suffers from the catastrophic forgetting issue, which is the tendency to forget previously learned knowledge when adapting to newly collected data. A plethora of state-of-the-art on-device continual learning works have been recently established, which seeks to stabilize the on-device continual learning process and further push forward the attainable accuracy on target task.

Parameter-Efficient Continual Fine-Tuning

Parameter-Efficient Fine-Tuning techniques still lack the ability to adjust models to multiple tasks continually due to catastrophic forgetting, and a 2025 survey on Parameter-Efficient Continual Fine-Tuning (PECFT) provides an overview of continual learning algorithms and PEFT methods.

Recent Approaches

The rise of foundation models has shifted the paradigm from training models from scratch to efficient fine-tuning via prompt-based methods, which substantially mitigates forgetting by freezing the backbone and updating only lightweight prompt parameters.

The search results reveal active research in this area, with multiple 2025 surveys and papers addressing these interconnected challenges of on-device learning, catastrophic forgetting mitigation, and efficient fine-tuning approaches.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

25. user (2026-06-14T08:24:59.671Z)

Web search results for query: "on-device fine-tuning vision language model consumer GPU feasibility 2025 QLoRA memory"

Links: [{"title": "Fine-Tuning Large Language Models (LLMs) Using QLoRA - GeeksforGeeks", "url": "https://www.geeksforgeeks.org/nlp/fine-tuning-large-language-models-llms-using-qlora/"}, {"title": "Frontiers | Resource-efficient fine-tuning of large vision-language models for multimodal perception in autonomous"}]

```
excavators","url":"https://www.frontiersin.org/journals/artificial-
intelligence/articles/10.3389/frai.2025.1681277/full"},"{"title":"ROMA: a Read-Only-Memory-based
Accelerator for QLoRA-based On-Device
LLM","url":"https://arxiv.org/pdf/2503.12988"},"{"title":"QLoRA - How to Fine-Tune an LLM on a
Single GPU | Towards Data Science","url":"https://towardsdatascience.com/qlora-how-to-fine-tune-
an-llm-on-a-single-gpu-4e44d6b5be32/"}, {"title":"Profiling LoRA/QLoRA Fine-Tuning Efficiency on
...", "url":"https://arxiv.org/pdf/2509.12229"}," {"title":"How can I fine-tune large language
models on a budget using LoRA and QLoRA on cloud
GPUs?","url":"https://www.runpod.io/articles/guides/how-to-fine-tune-large-language-models-on-a-
budget"}," {"title":"Evolution of meta's llama models and parameter-efficient fine-tuning of large
language models: a survey","url":"https://arxiv.org/pdf/2510.12178"}," {"title":"Resource-
efficient fine-tuning of large vision-language models for multimodal perception in autonomous
excavators","url":"https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12669176/"}, {"title":"EMLoC:
Emulator-based Memory-efficient Fine-tuning with LoRA
Correction","url":"https://arxiv.org/pdf/2506.12015"}]]
```

I've completed the web search for your query about on-device fine-tuning of vision language models on consumer GPUs using QLoRA. Here are the key findings:

Memory Efficiency and Feasibility

QLoRA makes the fine-tuning of large vision-language models (LVLMs) feasible on consumer-grade GPUs without compromising model accuracy or generalization capability. By combining quantization with LoRA, QLoRA significantly reduces VRAM usage making fine-tuning feasible on consumer-grade GPUs.

Memory Reduction

QLoRA can reduce memory usage by up to 75-80% compared to LoRA alone. For example, where LoRA might need ~20 GB for a 7B model, QLoRA could bring it down to ~8-10 GB. A 10B model can be fine-tuned with just 11.6GB of memory, which can easily run on consumer hardware like the free T4 GPU on Google Colab.

Vision-Language Model Applications

Recent research in 2025 demonstrates practical applications: A resource-efficient framework for fine-tuning LVLMs tailored to autonomous excavator operations leverages QLoRA in conjunction with the Unsloth framework, substantially reducing memory consumption and accelerating fine-tuning compared with conventional approaches.

Consumer GPU Benchmarks

LoRA/QLoRA fine-tuning on an RTX 4060 is feasible, with batch sizes up to 2 and sequence lengths up to 2048 tokens, with fp16 and PagedAdamW optimizers being the most efficient configuration.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

26. assistant (2026-06-14T08:25:02.708Z)

Let me ground two more specific claims: per-user/personalized model adaptation feasibility (the actual personalization architecture) and the small-N statistics problem for per-user A/B selection.

27. user (2026-06-14T08:25:11.073Z)

Web search results for query: "model selection overfitting small validation set cross-validation noise variance few samples"

Links: [{"title":"Overfitting, Model Tuning, and Evaluation of Prediction Performance - Multivariate Statistical Machine Learning Methods for Genomic Prediction - NCBI Bookshelf","url":"https://www.ncbi.nlm.nih.gov/books/NBK583970/"}, {"title":"Galactic Double

Neutron Star total masses and Gaussian mixture model selection", "url": "https://arxiv.org/pdf/1808.01129"}, {"title": "Overfitting, Model Selection, Cross Validation, Bias-Variance", "url": "https://people.cs.umass.edu/~domke/courses/sml/02overfitting.pdf"}, {"title": "Part II. Model Evaluation: Cross Validation, Bias and Variance Tradeoff and How to Diagnose Overfitting | by Ani Karenovna K | Medium", "url": "https://medium.com/@karenovna.ak/part-ii-evaluating-a-predictive-model-cross-validation-and-bias-and-variance-tradeoff-9874b836cd2e"}, {"title": "The importance of choosing a proper validation strategy in predictive models. Part 2: Recipes for (avoiding) overfitting-A tutorial - ScienceDirect", "url": "https://www.sciencedirect.com/science/article/pii/S0003267025012322"}, {"title": "3.1. Cross-validation: evaluating estimator performance ? scikit-learn 1.9.0 documentation", "url": "https://scikit-learn.org/stable/modules/cross_validation.html"}, {"title": "Model Validation: Problem Areas and Solutions - Overfitting and Underfitting | Rocketloop", "url": "https://rocketloop.de/en/blog/model-validation-overfitting-underfitting/"}, {"title": "0% found this document useful (0 votes)", "url": "https://www.scribd.com/document/270169773/Data-mining-and-model-selection"}]

Based on the search results, here are key findings related to model selection, overfitting, and validation with small samples:

Effects of Noise, Variance, and Small Sample Sizes

When data are very noisy, when sample sizes are small, or when the range of variables is limited, simpler underspecified models can actually produce more accurate predictions than more complex models. This highlights an important trade-off in model selection.

Cross-Validation as a Model Selection Strategy

Cross-validation (CV) is a strategy for model selection or algorithm selection. Overly complex models are expected to get over-fit to the training subsets and then provide inferior prediction performance on the test subsets.

Challenges with Small Datasets

A critical issue with small validation sets is that numerical cross-validation scores turn out not to be useful for small data sets, as the variance is too large to make any robust statements. This suggests that standard cross-validation approaches may be unreliable when working with limited data.

Overfitting and Model Complexity

When a model has too many features, it is prone to overfitting ? the phenomenon in which a predictive model fits itself to noise in the data, and models that overfit are too complex. Additionally, aggressive hyperparameter tuning can lead to overfitting where the model becomes overly specialized to the training data, occurring when the optimization process prioritizes reducing training error over validation performance, causing the model to focus on noise.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

28. user (2026-06-14T08:25:12.352Z)

Web search results for query: "personalized lightweight adapter per-user model adaptation frozen backbone test-time small data 2025"

Links: [{"title": "[2512.22398] Lightweight Inference-Time Personalization for Frozen Knowledge Graph Embeddings", "url": "https://arxiv.org/abs/2512.22398"}, {"title": "Mod-Adapter: Tuning-Free and Versatile Multi-concept Personalization via Modulation Adapter", "url": "https://arxiv.org/pdf/2505.18612"}, {"title": "Lightweight Inference-Time Personalization for Frozen Knowledge Graph", "url": "https://arxiv.org/pdf/2512.22398"}, {"title": "Lightweight Inference-Time Personalization for Frozen Knowledge Graph"}]

Embeddings", "url": "https://arxiv.org/html/2512.22398"}, {"title": "(PDF) Lightweight Inference-Time Personalization for Frozen Knowledge Graph Embeddings", "url": "https://www.researchgate.net/publication/399175280_Lightweight_Inference-Time_Personalization_for_Frozen_Knowledge_Graph_Embeddings"}, {"title": "Data-Efficient Adaptation", "url": "https://www.emergentmind.com/topics/data-efficient-adaptation"}, {"title": "Trainable Adapters: Efficient Model Adaptation", "url": "https://www.emergentmind.com/topics/trainable-adapter"}, {"title": "Beyond Retrieval: Learning Compact User Representations for Scalable LLM Personalization", "url": "https://arxiv.org/html/2606.04547"}, {"title": "Lightweight User-Personalization Method for Closed Split Computing", "url": "https://arxiv.org/pdf/2603.14958"}]

Based on the search results, I found several relevant papers and resources on lightweight adapter methods for model adaptation with frozen backbones:

Key Findings

****GatedBias (December 2025)****

GatedBias is a lightweight inference-time personalization framework that adapts frozen KG embeddings to individual user contexts without retraining or compromising global accuracy. The approach introduces structure-gated adaptation: profile-specific features combine with graph-derived binary gates to produce interpretable, per-entity biases, requiring only approximately 300 trainable parameters.

****Adapter-Based Approaches****

Several adapter-based methods employ lightweight modules for task-specific adaptation while keeping the foundation model weights frozen. Trainable adapters are lightweight, task-specialized modules inserted into frozen neural network backbones that employ small residual blocks?like bottleneck MLPs, convolutions, or attention modules?to modify feature representations while preserving core, generic knowledge.

****Small Data and Efficient Adaptation****

Unidirectional Thin Adapter (UDTA) architectures use small, bottlenecked blocks fed by intermediate backbone features, trained while freezing the backbone, delivering up to approximately 86% backward compute reduction, making adaptation feasible on small datasets or with compute-constrained hardware.

****Per-User Personalization****

Recent methods include PROPER which trains a dedicated LoRA per user, PER-PCS which reassembles shared adapter pieces, and P2P which generates adapters from profiles via a hypernetwork.

These approaches represent the current state-of-the-art in parameter-efficient, frozen-backbone adaptation with minimal data requirements.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

29. assistant (2026-06-14T08:25:15.637Z)

I now have strong grounding. Let me do one final targeted search on the on-device VLM inference (not just fine-tuning) feasibility on hobbyist hardware, since the repo already flags that as unmeasured ? this is the cold-start "batteries" delivery vehicle.

30. user (2026-06-14T08:25:25.311Z)

Web search results for query: "Qwen3-VL 4B quantized on-device inference consumer GPU 8GB video tokens latency 2025"

Links: [{"title": "Private LLM Inference on Consumer Blackwell GPUs: A Practical Guide for Cost-Effective Local Deployment in SMEs", "url": "https://arxiv.org/pdf/2601.09527"}, {"title": "MiniCPM-

SALA: Hybridizing Sparse and Linear Attention for Efficient Long-Context Modeling", "url": "https://arxiv.org/pdf/2602.11761"}, {"title": "Rethinking Model Efficiency: Multi-Agent Inference with Large Models", "url": "https://arxiv.org/pdf/2604.04929"}, {"title": "wangkanai/qwen3-vl-8b-instruct · Hugging Face", "url": "https://huggingface.co/wangkanai/qwen3-vl-8b-instruct"}, {"title": "OmniVinci: Enhancing Architecture and Data for Omni-Modal Understanding LLM", "url": "https://arxiv.org/pdf/2510.15870"}, {"title": "Qwen 3: What You Need to Know - Gradient Flow", "url": "https://gradientflow.com/qwen-3/"}, {"title": "Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency", "url": "https://arxiv.org/pdf/2504.03360"}, {"title": "Viability and Performance of a Private LLM Server for SMBs: A Benchmark Analysis of Qwen3-30B on Consumer-Grade Hardware", "url": "https://arxiv.org/html/2512.23029v1"}]

Based on the search results, here's what I found about Qwen3-VL 4B quantized on-device inference on consumer GPUs:

Qwen3-VL 4B Performance Metrics

The Qwen3-VL 4B model's vision preprocessing latency ranges from 0.91 ms at 256² resolution to 343.78 ms at 4096² resolution.

For inference latency with images, the Time-to-First-Token latency for Qwen3-VL 4B with a 1024² image and 512 query tokens was 39.02 ms.

Quantization and Memory Optimization

4-bit quantization (Q4) is generally considered effective with minimal performance loss, approximately halving the VRAM needed compared to 8-bit versions. This makes it viable for running on 8GB consumer GPUs through aggressive quantization approaches.

Consumer GPU Deployment

The search results indicate that on RTX 5090 (32GB VRAM), the baseline Qwen3-8B hits memory constraints at 256K tokens in quantized settings, while models using optimized attention mechanisms can scale to 1024K context lengths. For 8GB GPUs, even more aggressive quantization would be necessary.

The results focus primarily on larger Qwen3 models and video processing for multimodal variants, with less specific data available for the exact 4B quantized variant on 8GB consumer GPUs with video token handling at specific latency benchmarks from 2025.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

31. assistant (2026-06-14T08:25:35.237Z)

I have everything I need. Let me synthesize the technical-feasibility analysis of per-user continual personalization, grounded in both the repo's actual substrate and the web research.

Key structural findings from the code/docs:

- The "decision vs detection" split (`EXPERIMENT\_HARNESS.md` §2) is real and shipped ? `experiment.py::compare`, persisted candidates, pure offline re-scoring. This is the genuine enabler.
- The learned arbiter is a `LogisticRegression` (numpy, ~5 features) ? kilobyte JSON, trains in milliseconds. The TAL head is ~1M params, trains in minutes at \$0. These ARE personalizable cheaply.
- Qwen3-VL is the heavyweight; on-device fine-tuning of it is explicitly unmeasured even for inference on 8GB (`OWNED\_MODEL\_\*` open risks).
- The n=6 LOVO already exhibits the exact small-N pathology the pivot would face per-user (the first A/B's `testlarge\_short ? 0.000` threshold artifact; the ±0.242 spread; the bootstrap-CI + sign-test machinery in `eval\_stats.py` exists precisely because a bare mean lies at small N).

- The correction loop substrate exists structurally (``candidate_segments.human_label``, ``confirmed_segment_id``) but is a per-window confirm field, not a wired per-user training-label feedback pipeline.

32. user (2026-06-14T08:27:20.011Z)

Structured output provided successfully

33. assistant (2026-06-14T08:27:36.256Z)

I've completed the technical-feasibility analysis and returned it via StructuredOutput.

Summary of what I found, grounded in the repo's actual substrate plus web research:

The pivot is feasible-now at the cheap decision layer and unproven/needs-research at the heavy VLM layer. The owner's belief that the harness was designed to enable this is structurally correct ? but only for the lightweight layer.

Key load-bearing evidence from the code:

- ``backend/eval/experiment.py::compare`` + the persisted-candidate substrate implement exactly the "detect once, re-score per-user offline" architecture the pivot needs. Per-user cue ON/OFF is ``compare()`` + ``eval_stats.paired_delta_ci``.
- The personalizable artifacts are genuinely tiny:
``backend/eval/classifier.py::LogisticRegression`` (~5 features, kilobyte JSON, ms training) and the ~1.1M-param Gen-0 head (\$0, minutes on a 2070 Super).
- The heavyweight ``Qwen3-VL`` cannot be QLoRA'd on the 8GB box (``OWNED_MODEL_IMPLEMENTATION_PLAN.md``); even quantized on-device inference on 8GB is an explicitly unmeasured open risk.

The dominant risk is the small-N statistics problem, and it is strictly worse per-user than the n=6 corpus problem the repo already grapples with ? the entire ``backend/eval/eval_stats.py`` module exists because a bare LOVO mean lies at small N, and the first real A/B (``docs/archives/quality-iterations/2026-06-first-ab-experiment/README.md``) already exhibited the failure mode concretely (learned variant collapsed to F1=0.000 on the 6-rally video via a threshold artifact).

Catastrophic forgetting is bifurcated: safe for the frozen-WASB + tiny-head cheap layer (an established forgetting-resistant pattern), but re-emerges at the "batteries upgrade" seam if the shipped baseline is a continually-fine-tuned VLM.

Two missing engineering seams: the correction-loop ? per-user-label ? retrain pipeline (DB columns exist at ``database.py:109-110`` but the flow isn't wired, and the only wired label source, Gemini silver, is licensing-forbidden as a training source), and a per-user no-regression guard (the existing gate assumes a shared golden baseline a deliberately-overfit per-user model has no analogue for).

Relevant files: ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\experiment.py``, ``...\backend\eval\classifier.py``, ``...\backend\eval\eval_stats.py``, ``...\backend\eval\calibration.py``, ``...\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md``, ``...\docs\MULTI_SIGNAL_FUSION_PLAN.md``, ``...\docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md``, ``...\docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md``.

Note: ``git pull --ff-only`` failed ("Cannot fast-forward to multiple branches"), so this analysis is read-only on local state and may lag remote master.